Lec07：结构型设计模式

Structural Design Patterns

课程编号：22WHSE22006（2023版培养计划）

授课对象：23 级软件工程、服务科学与工程专业

主讲教师：辛国栋

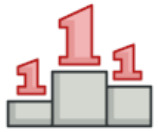
Email：xinguodong@163.com



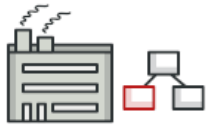
QQ 群：软件构造2025

群　号：311528424

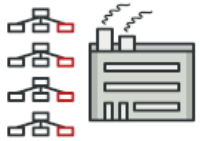
创建型模式(5 种)
Creational Patterns



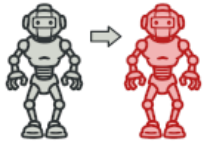
单例
Singleton



工厂方法
Factory Method



抽象工厂
Abstract Factory



原型
Prototype



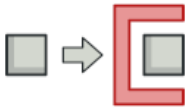
建造者
Builder



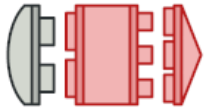
编外：简单工厂
Simple Factory

对象产生与使用解耦

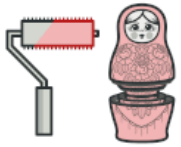
结构型模式(7 种)
Structural Patterns



代理
Proxy



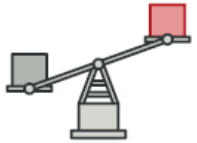
适配器
Adapter



装饰
Decorator



桥接
Bridge



享元
Flyweight



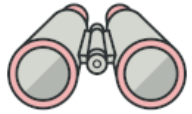
组合
Composite



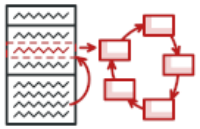
外观
Facade

功能解耦

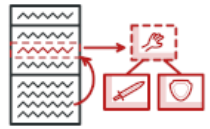
行为型模式（11 种)
Behavioral Patterns



观察者
Observer



状态
State



策略
Strategy



模版方法
Template Method



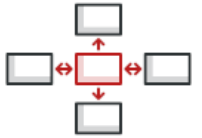
访问者
Visitor



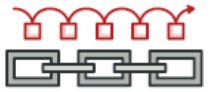
解释器
Interpreter



备忘录
Memento



中介
Mediator



迭代器
Iterator



责任链
Chain of Responsibility



命令
Command

行为解耦

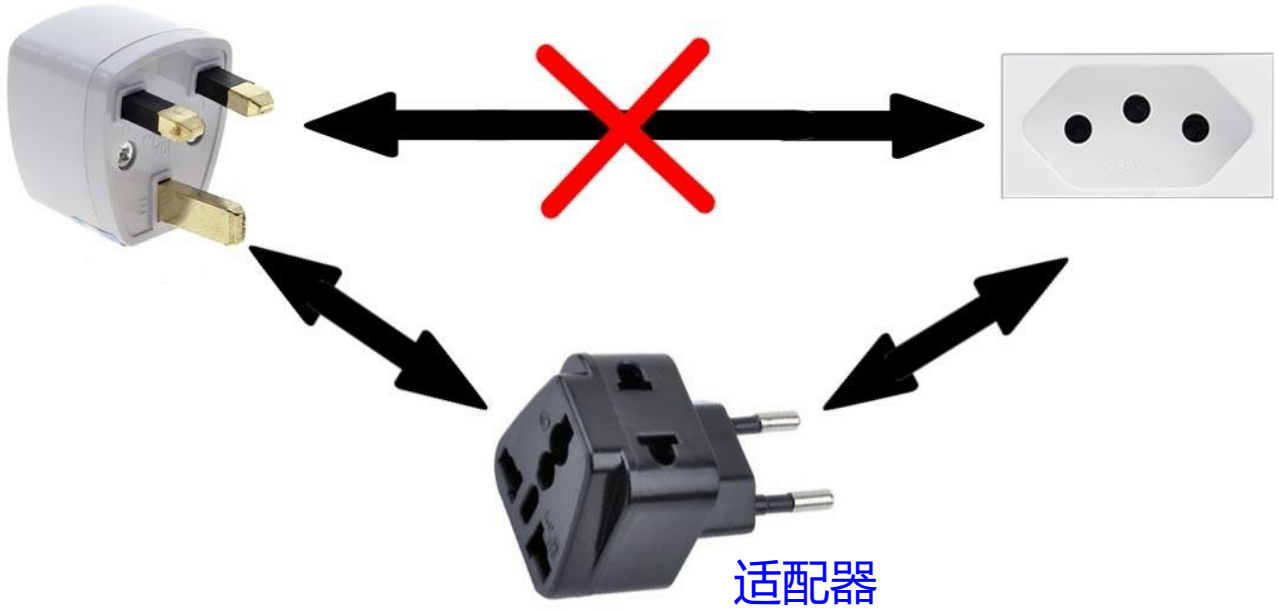
引言：七种结构型模式的具体作用

No	结构型模式	作用
1	适配器模式 Adapter Pattern	两个不兼容的接口之间互联互通
2	代理模式 Proxy Pattern	不改变原始类的情况下，为其附加功能
3	桥接模式 Bridge Pattern	存在两个独立变化的维度，让这两个维度可以独立进行扩展
4	装饰器模式 Decorator Pattern	不改变原始类接口的情况下，对原始类功能进行增强
5	组合模式 Composite Pattern	相似对象进行组合，形成树形结构
6	享元模式 Flyweight Pattern	通过共享对象内部状态减少内存消耗和提升系统性能
7	外观模式 Facade Pattern	封装复杂性与统一访问入口

1. 适配器模式

Adapter Design Pattern

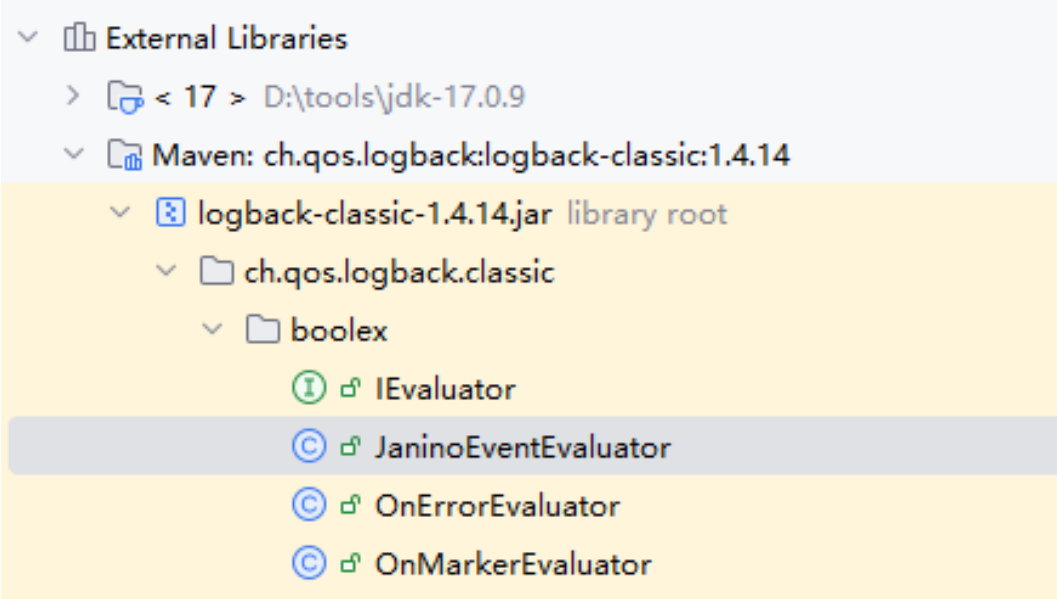
1.1 什么是适配器?

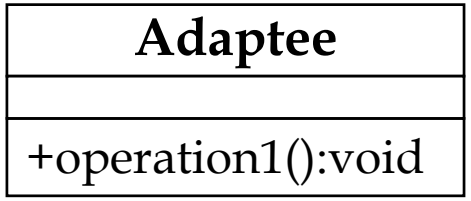


顾名思义，适配器模式就是用来做适配的，它将不兼容的接口转换为可兼容的接口，让原本由于接口不兼容而不能一起工作的类可以一起工作

1.2 引例

- 某项目中，假设甲方提供的驱动 jar 包中有一个 Adaptee 的类，该类有一个返回值为void 的方法 operation1()，如下类图所示：

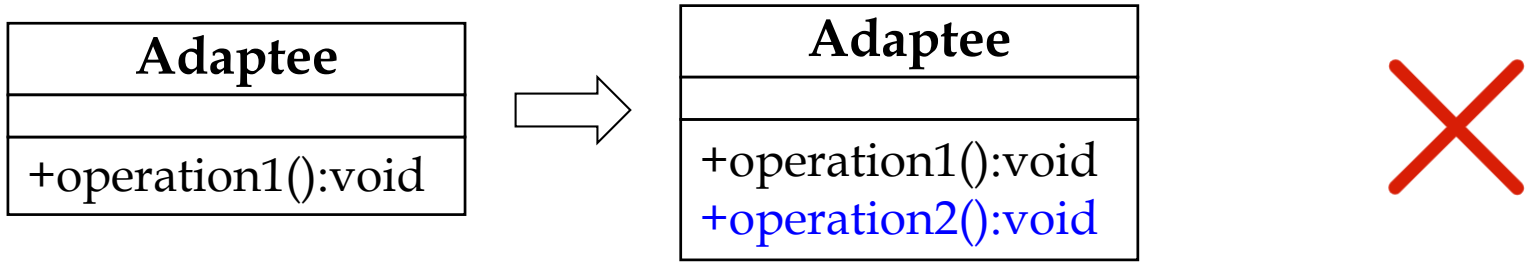




- 在使用该类的过程中，发现其功能不足。该类还应该提供一个 operation2() 才能满足使用要求
- 怎样解决该问题?

1.3 解决方案讨论-不可能在给出的类中加功能

- 在类 Adaptee 中添加方法 operation2()?

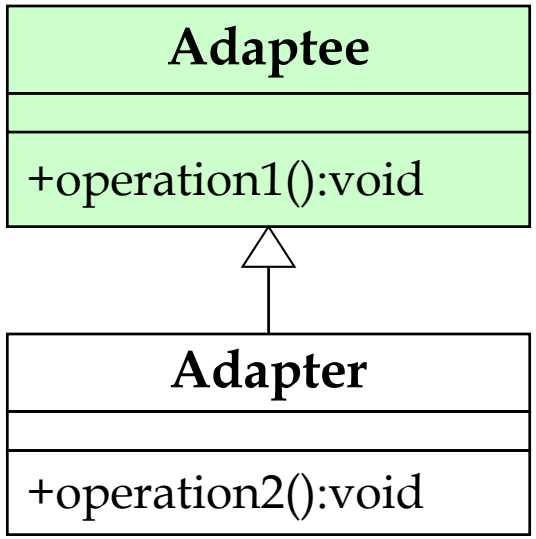


  - Adaptee 类没有源代码，作为用户，没办法修改甲方给的 jar 包。
  - 即便有源代码，直接在已发布的类中添加方法，也不是一个好的做法。
  - 如果 Adaptee 类已经被其它应用程序使用，也要考虑改变之后的副作用

1.3 解决方案讨论-尝试通过继承解决问题

- 解决方案1：新建一个子类 Adapter 继承 Adaptee，并在该子类中添加 operation2() 方法。

原始类



扩展类

这样甲方提供的 Adaptee 就提升成了一个接口（指在设计层面的逻辑顶层）。超类不会知道任何的子类信息，因此不会有任何修改，满足实际情况

子类 Adapter 拥有继承自超类中的 operation1() 方法，和自身新增的operation2() 方法。

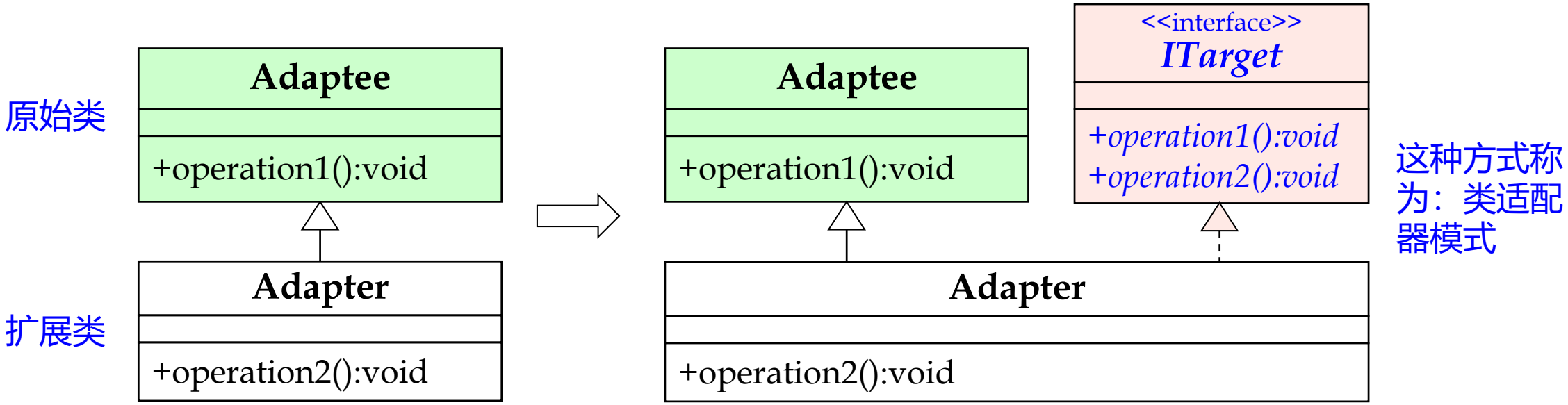
```
Adaptee adaptee = new Adapter(); //以接口声明对象，具体类实例化
adaptee.operation2(); //错误，无法获取超类中未声明的方法
```

能够解决当前问题，但此设计不利于扩展

错误原因：Java 语言是静态编译语言，编译时决定了该类型具有的属性和方法，因此以父类声明对象，该对象只有找到父类中有的方法；

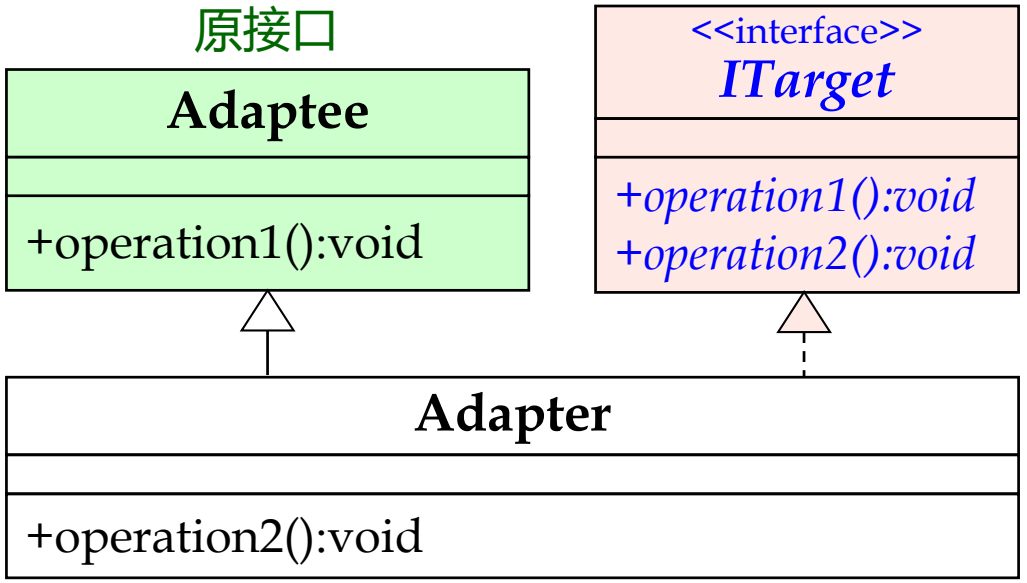
1.3 解决方案讨论-尝试通过继承解决问题

- 父类中没有声明 operation2()，又不可能在父类加上operation2()。如何解决此问题?
- 那么我们是否可以给子类加个父类或者换个父类?
- 解决方案2：在继承的基础上，为子类增加父类



Adapter 类需要实现 ITarget 接口中所有的的方法。尽管 Adapter 类中没有显示 operation1()，但根据继承的原理，该方法自动从 Adaptee 继承，因此只需要实现 operation2() 即可。

1.3 解决方案讨论-增加父类/父接口




```java
public class Adaptee { //甲方提供给我们的类
    public void operation1() {
        System.out.print("operation1");
    }
}
```

```java
public interface ITarget { //新接口
    void operation1();
    void operation2();
}
```

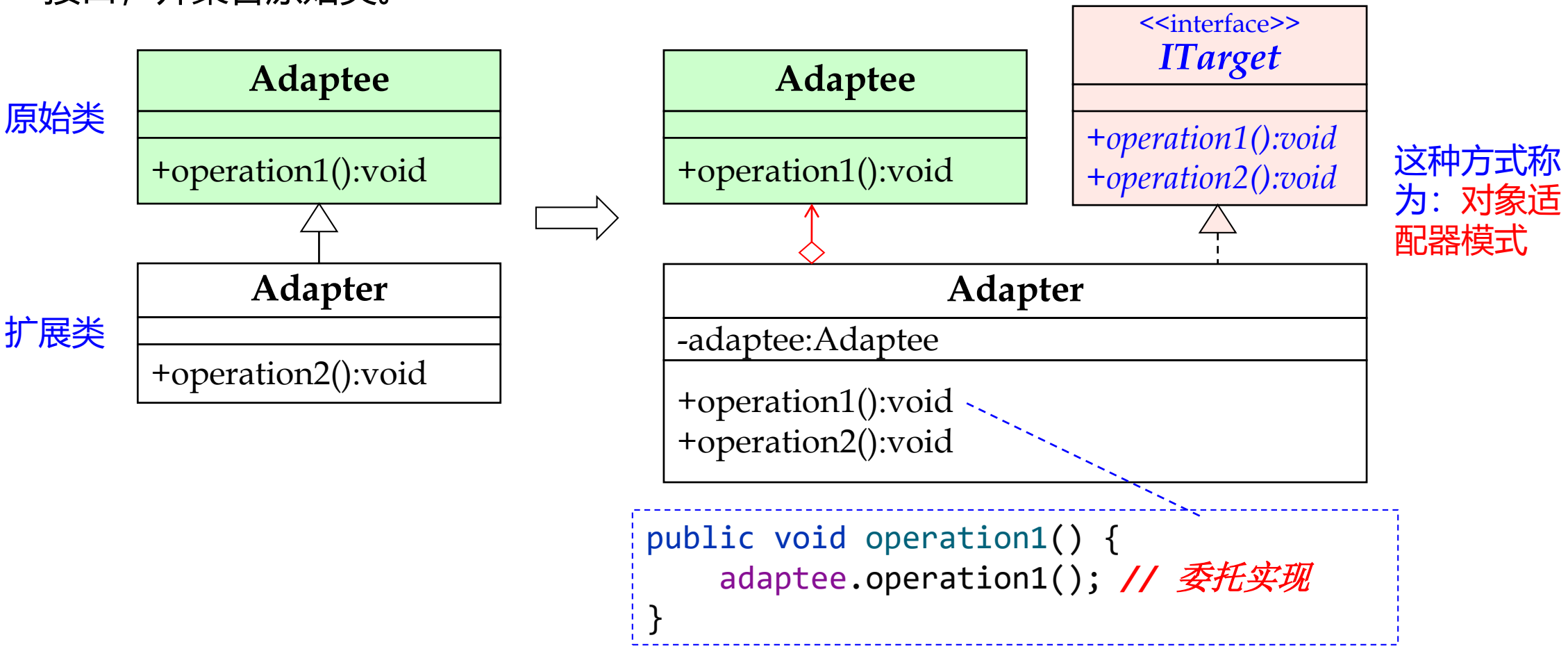
```java
public class Client {
    public static void main(String[] args) {
        Target adp = new Adapter();
        adp.operation1();
        adp.operation2();
    }
}
```

```java
//适配器类
public class Adapter extends Adaptee implements Target {
    public void operation2() {
        // 写代码实现此方法
        System.out.println("operation2");

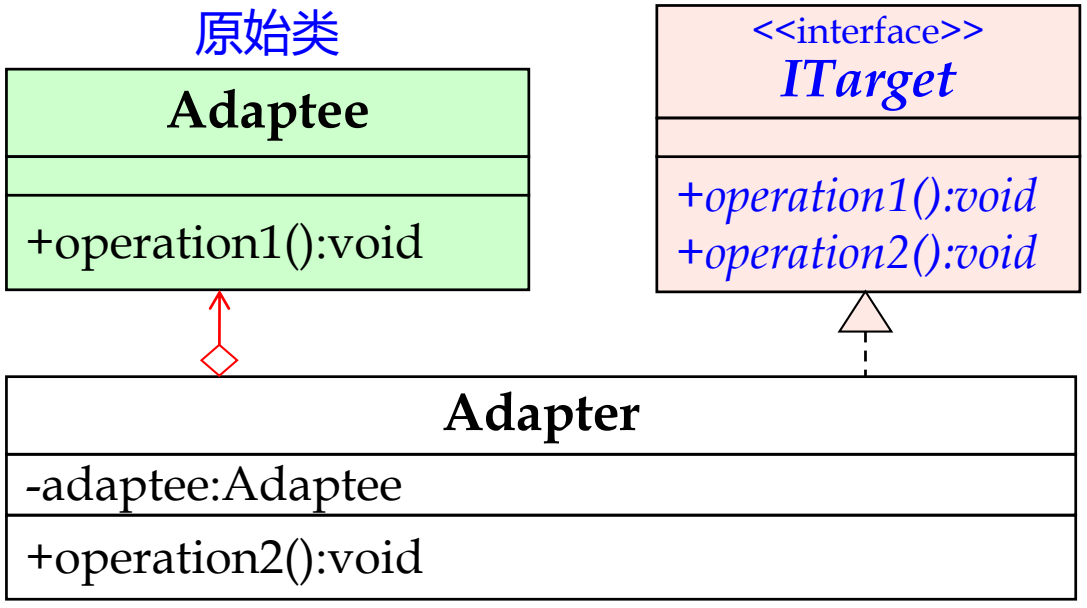
    }
}
```

1.3 解决方案讨论-换个父类

- 解决方案3：为子类换个父类。适配器类 Adapter 不再继承 Adaptee，而是实现新建的 Itarget 接口，并聚合原始类。



1.3 解决方案讨论-换个父类




```
public class Client {
    private static Target adp;
    public static void main(String[] args) {
        adp = new Adapter(new Adaptee());
        adp.operation1();
        adp.operation2();
    }
}
```

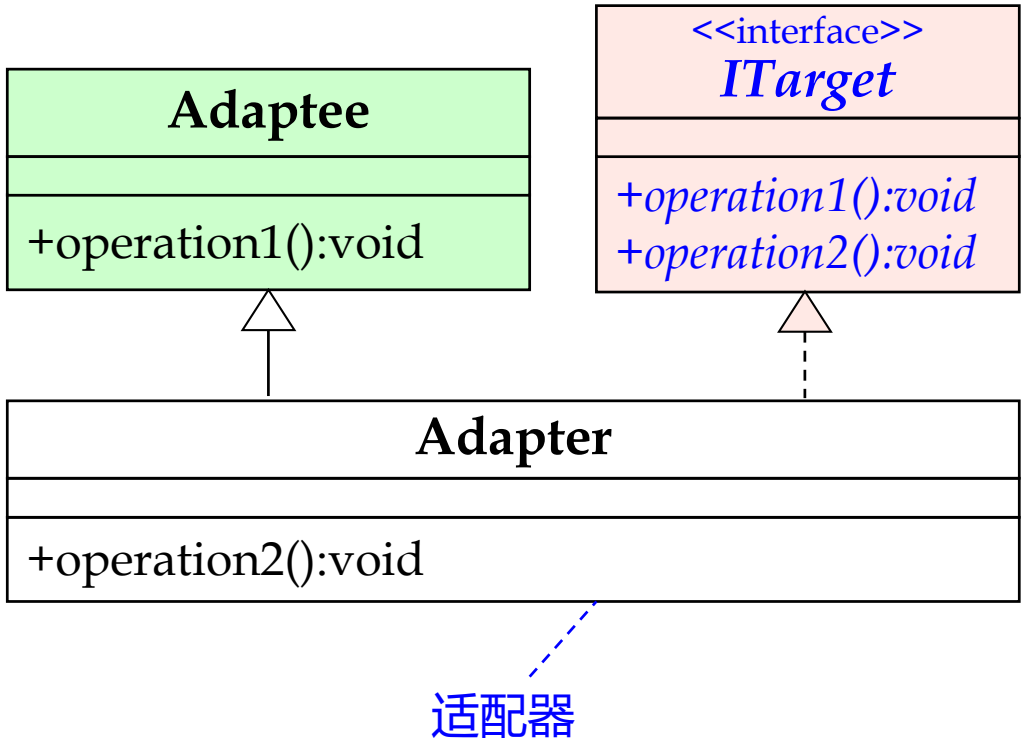
```
//适配器类
public class Adapter implements Target {
    private Adaptee adaptee;

    public Adapter(Adaptee adaptee) {
        this.adaptee = adaptee;
    }

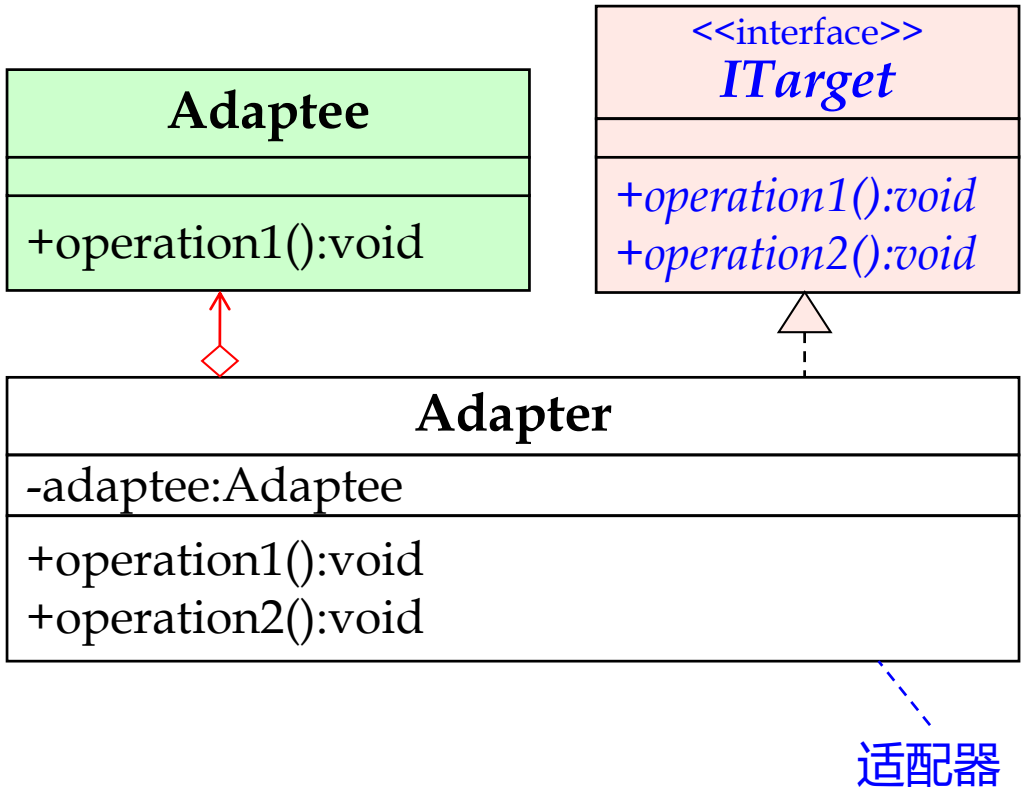
    @Override
    public void operation1() {
        adaptee.operation1(); // 委托实现
    }

    public void operation2() {
        // 写代码实现此方法
        System.out.println("operation2");
    }
}
```

1.4 类适配器模式与对象适配器模式对比



类适配器方式：
定义一个适配器类来实现当前系统的业务接口
同时又继承现有组件库中已经存在的组件



对象适配器方式：
将现有组件库中已经实现的组件引入适配器类中，
该类同时实现当前系统的业务接口。

1.5 适配器模式的应用场景-①封装有缺陷的接口设计

- 为了隔离设计上的缺陷，我们希望对外部系统提供的接口进行二次封装，抽象出更好的接口设计，这个时候就可以使用适配器模式

```
public class CD { //这个类来自外部 SDK，我们无权修改它的代码
    //...
    public static void staticFunction1() {  //... }
    public void uglyNamingFunction2() {  //... }
    public void tooManyParamsFunction3(int paramA, int paramB, ...) { //... }
    public void lowPerformanceFunction4() {  //... }
}
```

```
// 使用适配器模式进行重构
public interface ITarget {
    void function1();
    void function2();
    void fucntion3(ParamsWrapperDefinition paramsWrapper);
    void function4();
    //...
}
```

1.5 适配器模式的应用场景-①封装有缺陷的接口设计

```
//注意：适配器类的命名不一定非得末尾带Adaptor
public class CDAdaptor extends CD implements ITarget {
    @Override
    public void function1() {
        super.staticFunction1();
    }
    @Override
    public void function2() {
        super.uglyNamingFunction2();
    }
    @Override
    public void fucntion3(ParamsWrapperDefinition paramsWrapper) {
        super.tooManyParamsFunction3(paramsWrapper.getParamA(),paramsWrapper.getParamB());
    }
    @Override
    public void function4() {
        // ...reimplent it...
    }
}
```

1.5 适配器模式的应用场景-②统一多个类的接口设计

- 假设我们的系统要对用户输入的文本内容做敏感词过滤。
  - 为了提高过滤的召回率，我们引入了多款第三方敏感词过滤系统，依次对用户输入的内容进行过滤，过滤掉尽可能多的敏感词。
  - 但是，每个系统提供的过滤接口都是不同的。这就意味着我们没法复用一套逻辑来调用各个系统。
- 第三方敏感词给出的类如下：

1.5 适配器模式的应用场景-②统一多个类的接口设计

```
public class ASensitiveWordsFilter { // A敏感词过滤系统提供的接口
    //text是原始文本，函数输出用***替换敏感词之后的文本
    public String filterSexyWords(String text) {
        // ...
    }
    public String filterPoliticalWords(String text) {
        // ...
    }
}
```

```
public class BSensitiveWordsFilter { // B敏感词过滤系统提供的接口
    public String filter(String text) {
        //...
    }
}
```

```
public class CSensitiveWordsFilter { // C敏感词过滤系统提供的接口
    public String filter(String text, String mask) {
        //...
    }
}
```

1.5 适配器模式的应用场景-②统一多个类的接口设计

```
//未使用适配器模式之前的代码：代码的可测试性、扩展性不好
public class RiskManagement {
    private ASensitiveWordsFilter aFilter = new ASensitiveWordsFilter();
    private BSensitiveWordsFilter bFilter = new BSensitiveWordsFilter();
    private CSensitiveWordsFilter cFilter = new CSensitiveWordsFilter();

    public String filterSensitiveWords(String text) {
        String maskedText = aFilter.filterSexyWords(text);  //第一遍

        maskedText = aFilter.filterPoliticalWords(maskedText); //第二遍

        maskedText = bFilter.filter(maskedText); //第三遍

        maskedText = cFilter.filter(maskedText, “***”); //第四遍

        return maskedText; // 返回过滤后的文本
    }
}
```

接下来使用适配器模式进行改造：

1.5 适配器模式的应用场景-②统一多个类的接口设计

```
//使用适配器模式进行改造
public interface ISensitiveWordsFilter { //统一接口定义
    String filter(String text);
}
```

```
//按此形式可以设置多个适配器，适配到统一的接口上

public class ASensitiveWordsFilterAdaptor implements ISensitiveWordsFilter {
    private ASensitiveWordsFilter aFilter;

    public String filter(String text) {
        String maskedText = aFilter.filterSexyWords(text);
        maskedText = aFilter.filterPoliticalWords(maskedText);
        return maskedText;
    }
}

//...省略BSensitiveWordsFilterAdaptor、CSensitiveWordsFilterAdaptor...
```

1.5 适配器模式的应用场景-②统一多个类的接口设计

```
//扩展性更好，符合开闭原则，如果添加一个新的敏感词过滤系统，
//这个类完全不需要改动；而且基于接口而非实现编程，代码的可测试性更好。

public class RiskManagement {
    private List<ISensitiveWordsFilter> filters = new ArrayList<>();

    public void addSensitiveWordsFilter(ISensitiveWordsFilter filter) {
        filters.add(filter);
    }

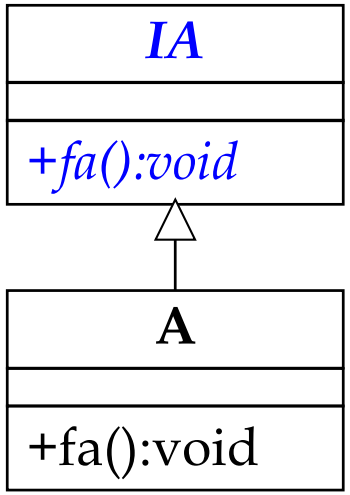
    public String filterSensitiveWords(String text) {
        String maskedText = text;
        for (ISensitiveWordsFilter filter : filters) {
            maskedText = filter.filter(maskedText);
        }
        return maskedText;
    }
}
```

1.5 适配器模式的应用场景-③替换依赖的外部系统

```java
// 外部系统 A
public interface IA {
    void fa();
}
```

```java
public class A implements IA {
    public void fa() {
        //...
    }
}
```

外部系统 A



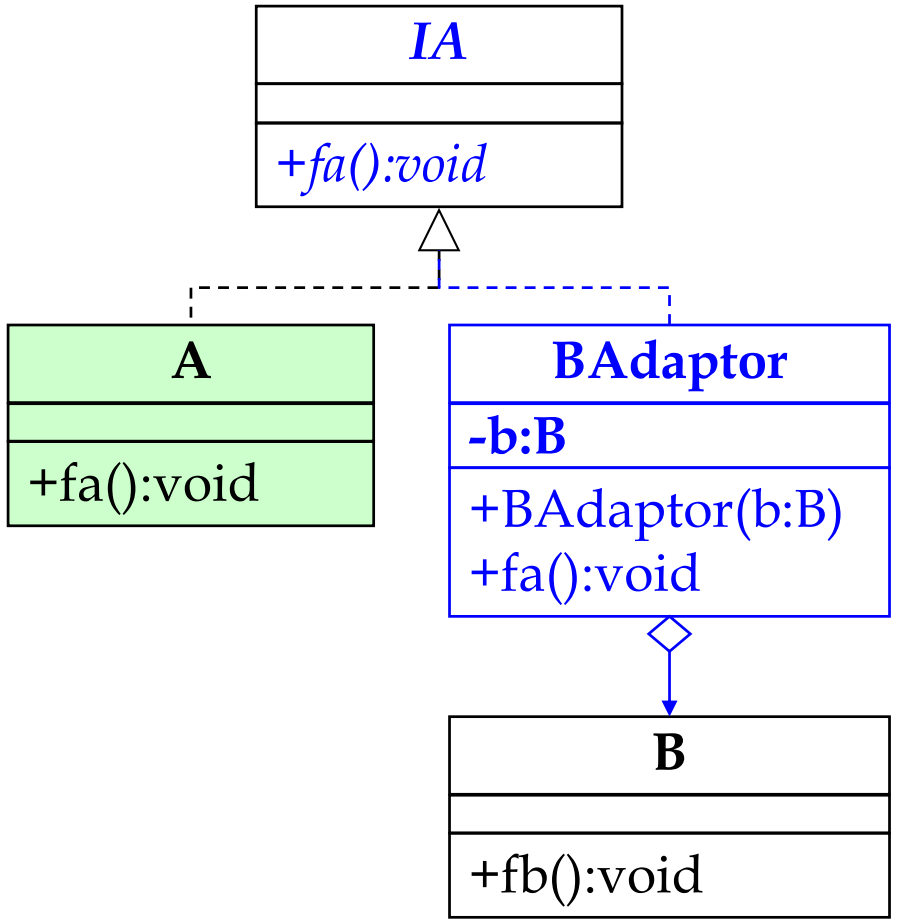

```java
// 在我们的项目中，外部系统 A 的使用示例
public class Demo {
    private IA a;

    public Demo(IA a) {
        this.a = a;
    }
    //...
}
```

```java
public class ClientA {
    public static void main(String[] args) {
        Demo d = new Demo(new A());
    }
}
```

如果想将外部系统 A 这套体系换成 外部系统 B，该如何处理?

1.5 适配器模式的应用场景-③替换依赖的外部系统



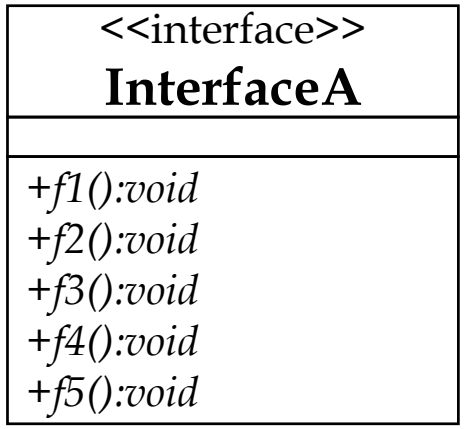

```java
public class BAdaptor implements IA {
    private B b;
    public BAdaptor(B b) {
        this.b = b;
    }
    public void fa() {
        //...
        b.fb();
    }
}
```

```java
public class ClentB {
    public static void main(String[] args) {
        Demo d = new Demo(new BAdaptor(new B()));
    }
}
```

还可以用在不同格式的数据之间的适配。例如 Java 中的 Arrays.asList() 也可以看作一种数据适配器，将数组类型的数据转化为集合容器类型。

1.6 适配器模式课堂练习

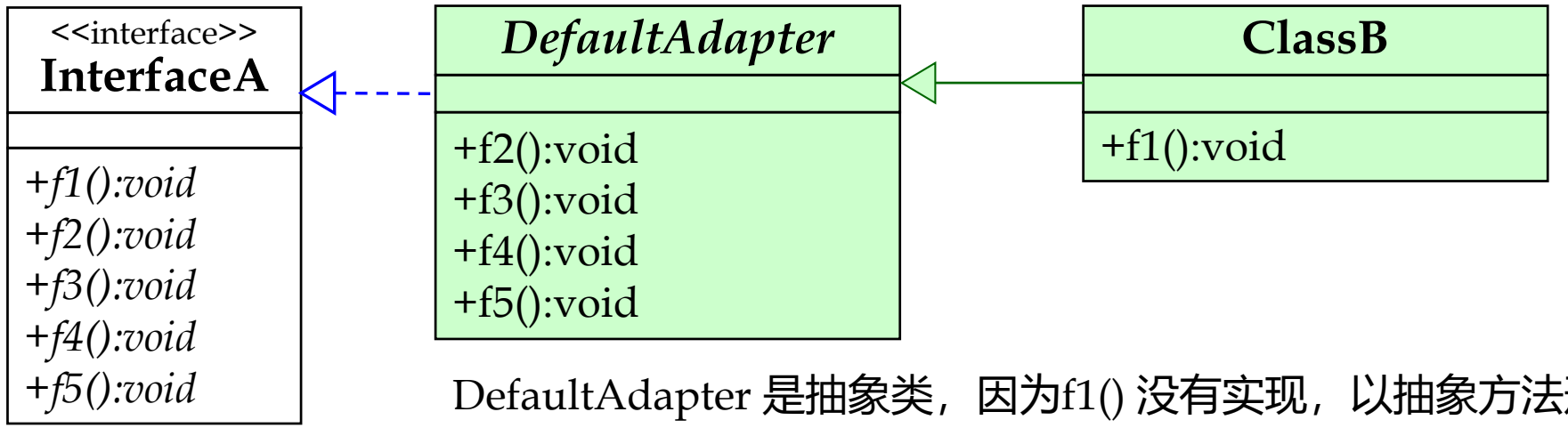
- 假设有一个接口，这个接口有五个方法，具体如下：




```
public class ClassB implements InterfaceA{
    @Override
    public void f1() {
        //编写实现代码
    }
}
```

- 你准备写一个 ClassB 实现这个接口，但是你只对接口所规定的五个方法中的 f1() 感兴趣。如果你把 ClassB 写成如左图代码所示，请问行不行？
- 如果不行，请给出一个可行的方案。

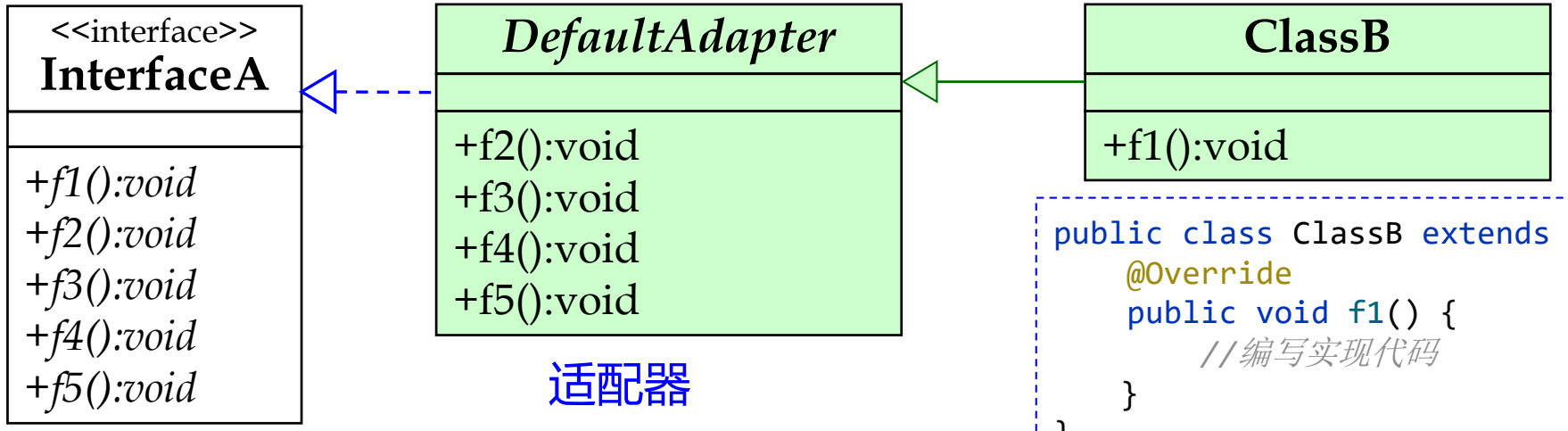
1.6 适配器模式课堂练习



DefaultAdapter 是抽象类，因为f1() 没有实现，以抽象方法形式继承

DefaultAdapter 继承/实现 InterfaceA，并且实现 f2() ~ f5()，但并不实现 f1()，此时 f1() 自动在 DefaultAdapter 中继承，并且在 DefaultAdapter 中依然是抽象的，因此 DefaultAdapter 是抽象类，需要用 abstract 修饰

1.6 适配器模式课堂练习




```
public class ClassB extends InterfaceAdaper{
    @Override
    public void f1() {
        //编写实现代码
    }
}
```

```
public interface InterfaceA {
    void f1();
    void f2();
    void f3();
    void f4();
    void f5();
}
```

```
public abstract class InterfaceAdaper implements InterfaceA {
    @Override
    public void f2() { }
    @Override
    public void f3() { }
    @Override
    public void f4() { }
    @Override
    public void f5() { }
}
```

2. 代理模式

Proxy Design Pattern

2.1 引例：性能计数器的应用重构

- 假设我们开发了一个性能计数器 MetricsCollector 类，用来收集接口请求的原始数据，比如访问时间、处理时长等。

```
public class MetricsCollector {
    private MetricsStorage metricsStorage;  // 基于接口而非实现编程，依赖注入。。。
    // 其他代码省略…
    public void recordRequest(RequestInfo requestInfo) {
        if (requestInfo == null || StringUtils.isBlank(requestInfo.getApiName())) {
            return;
        }

        // 信息采集后，存储于 requestInfo ，代码省略…

        metricsStorage.saveRequestInfo(requestInfo);
    }
}
```

- 在业务系统中，我们采用如下方式来使用这个 MetricsCollector 类:

```
public class UserController {

    //...省略其他属性和方法...
    private MetricsCollector metricsCollector; // 依赖注入

    public UserVo login(String telephone, String password) {
        long startTimestamp = System.currentTimeMillis();
        // ... 省略login逻辑...
        long endTimeStamp = System.currentTimeMillis();
        long responseTime = endTimeStamp - startTimestamp;
        RequestInfo requestInfo = new RequestInfo("login", responseTime, startTimestamp);
        metricsCollector.recordRequest(requestInfo);
        //...返回UserVo数据...
    }

    public UserVo register(String telephone, String password) {
        long startTimestamp = System.currentTimeMillis();
        // ... 省略register逻辑...
        long endTimeStamp = System.currentTimeMillis();
        long responseTime = endTimeStamp - startTimestamp;
        RequestInfo requestInfo = new RequestInfo("register", responseTime, startTimestamp);
        metricsCollector.recordRequest(requestInfo);
        //...返回UserVo数据...
    }
}
```

横切逻辑

横切逻辑

横切逻辑

横切逻辑

2.1 引例：性能计数器的应用重构

- 除了代码重复，当前的写法还有两个问题：

① 性能计数器 MetricsCollector 框架代码侵入到业务代码中，跟业务代码高度耦合。如果未来需要替换这个框架，那替换的成本会比较大（需要在 UserController 中更改代码，重新测试等）

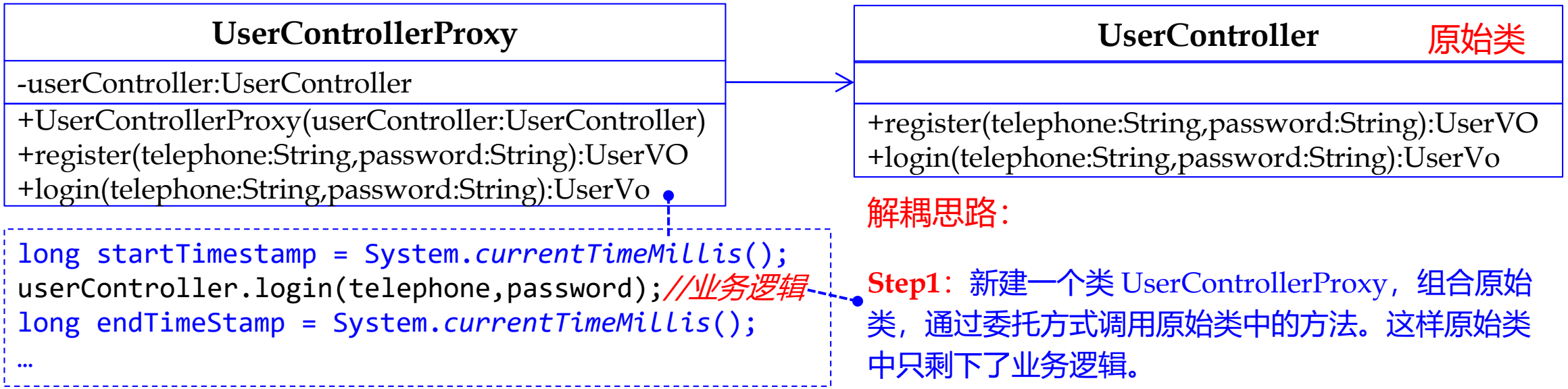
```
 public UserVo login(String telephone, String password) {
        long startTimestamp = System.currentTimeMillis();
业务逻辑 ⟶ // ... 省略 login逻辑...
        long endTimeStamp = System.currentTimeMillis();
        long responseTime = endTimeStamp - startTimestamp;
        RequestInfo requestInfo = new RequestInfo("login", responseTime, startTimestamp);
        metricsCollector.recordRequest(requestInfo);
业务逻辑 ⟶ //...返回 UserVo数据...
    }
```

② 收集接口请求的代码跟业务代码无关，本就不应该放到一个类中。业务类最好只聚焦业务处理

- 如何解决这两个问题？即：将性能计数器框架代码 和 业务代码解耦 ？

2.2 重构：将框架代码和业务代码解耦

```
 public UserVo login(String telephone, String password) {
需挪走{  long startTimestamp = System.currentTimeMillis();
        // ... 省略 login逻辑... 业务逻辑保留
需挪走{  long endTimeStamp = System.currentTimeMillis();
        long responseTime = endTimeStamp - startTimestamp;
        RequestInfo requestInfo = new RequestInfo("login", responseTime, startTimestamp);
        metricsCollector.recordRequest(requestInfo);
        //...返回 UserVo数据...业务逻辑保留
    }
```

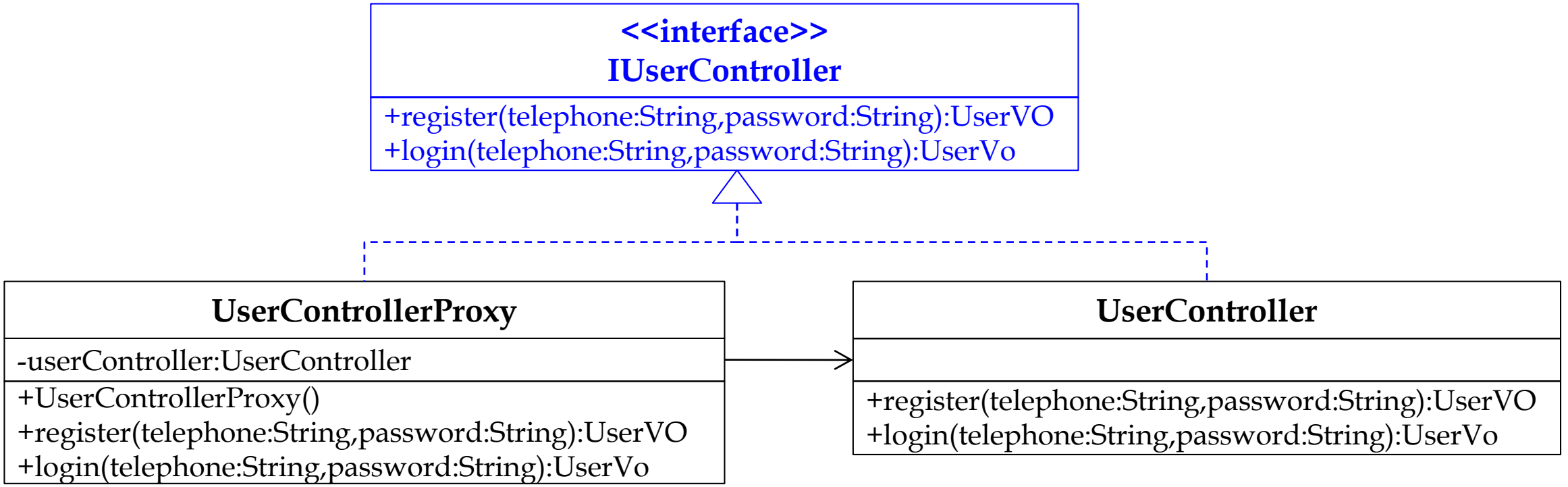


解耦思路：

Step1：新建一个类 UserControllerProxy，组合原始类，通过委托方式调用原始类中的方法。这样原始类中只剩下了业务逻辑。

2.2 重构：将框架代码和业务代码解耦

Step2：在 Step1 基础上，收集接口信息时，UserController 对象能被替换成 UserControllerProxy 对象。



使用示例：

```
// 将 UserController 类对象替换为 UserControllerProxy 类对象：超类声明，子类实例化。
IUserController userContr = new UserControllerProxy(new UserController());
```

2.2 重构：将框架代码和业务代码解耦

```
public interface IUserController {
    UserVo login(String telephone, String password);
    UserVo register(String telephone, String password);
}
```

```
public class UserController implements IUserController {
    //...省略其他属性和方法...
    @Override
    public UserVo login(String telephone, String password) {
        //...省略login逻辑...
        //...返回UserVo数据...
    }

    @Override
    public UserVo register(String telephone, String password) {
        //...省略register逻辑...
        //...返回UserVo数据...
    }
}
```

2.2 重构：将框架代码和业务代码解耦

```
public class UserControllerProxy implements IUserController {
    private MetricsCollector metricsCollector;
    private UserController userController;

    public UserControllerProxy(UserController userController) {
        this.userController = userController;
        this.metricsCollector = new MetricsCollector();
    }

    @Override
    public UserVo login(String telephone, String password) {
        long startTimestamp = System.currentTimeMillis();

        UserVo userVo = userController.login(telephone, password);

        long endTimeStamp = System.currentTimeMillis();
        long responseTime = endTimeStamp - startTimestamp;
        RequestInfo requestInfo = new RequestInfo("login", responseTime, startTimestamp);
        metricsCollector.recordRequest(requestInfo);
        return userVo;
    }
    //其他代码略...
}
```

增加的横切逻辑

原类的业务逻辑

增加的横切逻辑

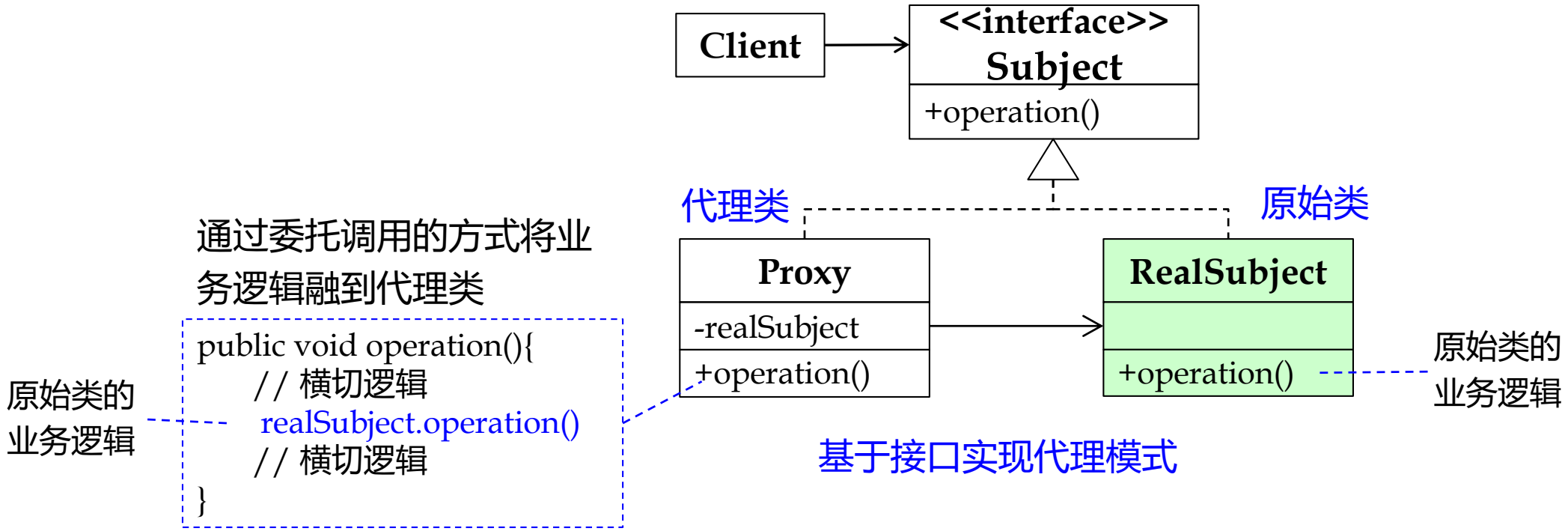
2.3 代理模式（Proxy Design Pattern）的概念

- 代理模式（Proxy Design Pattern）：在不改变原始类代码的情况下，通过引入代理类来给原始类附加功能

- 代理模式的典型应用：Spring framework AOP（面向切面编程）

2.4 静态代理的两种实现-①基于接口实现代理模式

- 如果原始类代码由我们开发维护，可以采用接口实现方式。



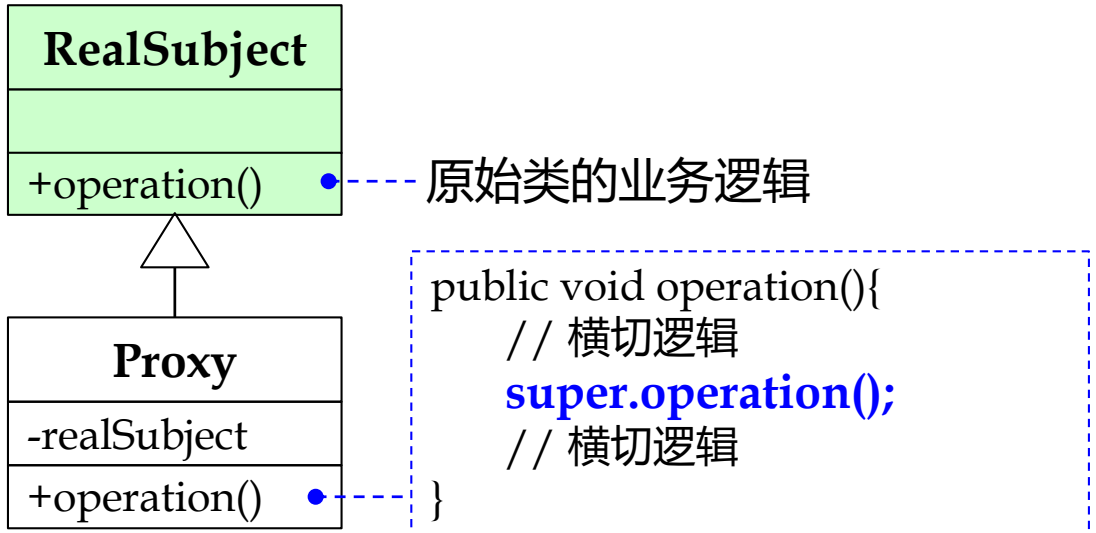
引例中，我们将框架代码和业务代码解耦的方式，就是基于接口实现的静态代理

2.4 静态代理的两种实现-②基于继承实现代理模式

- 如果我们对原始类代码无修改权限（例如第三方类库），此时一般采用继承实现

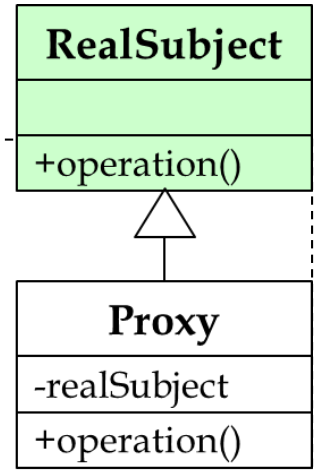
在子类中覆写父类的方法，添加横切逻辑。

在编程过程中，用子类替换父类



基于继承实现代理模式

2.4 静态代理的两种实现-②基于继承实现代理模式




```
public class UserControllerProxy extends UserController {
    private MetricsCollector metricsCollector;

    public UserControllerProxy() {
        this.metricsCollector = new MetricsCollector();
    }
    @Override
    public UserVo login(String telephone, String password) {
        long startTimestamp = System.currentTimeMillis();                    增加的横切逻辑

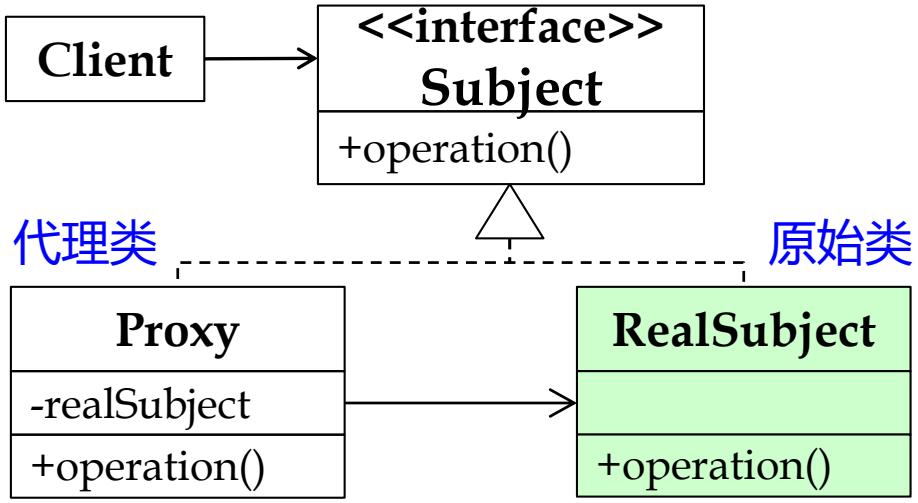
        UserVo userVo = super.login(telephone, password);                    原类的业务逻辑

        long endTimeStamp = System.currentTimeMillis();
        long responseTime = endTimeStamp - startTimestamp;
        RequestInfo requestInfo = new RequestInfo("login", responseTime, startTimestamp);
        metricsCollector.recordRequest(requestInfo);                         增加的横切逻辑

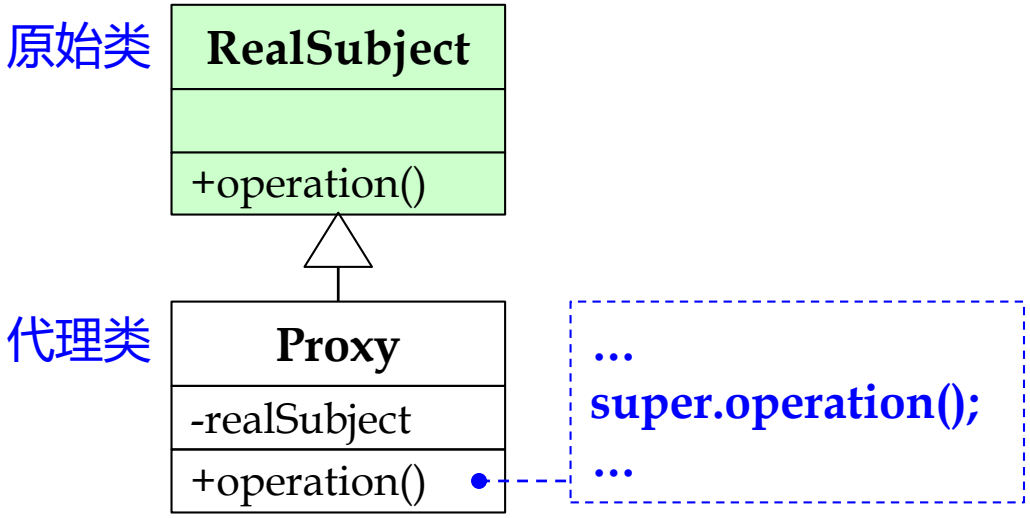
        return userVo;
    }
    // 其他代码省略
}
```

2.4 静态代理-缺点讨论

- 静态代理存在较严重的问题：



基于接口实现代理模式



基于继承实现代理模式

  - 需要针对每个类都创建一个代理类（因此叫做静态代理）。导致项目中类的个数成倍增加，增加了代码维护成本
  - 在代理类中，将原始类中所有的方法，都重新实现一遍，并且为每个方法都附加相似的代码逻辑，增加了不必要的开发成本

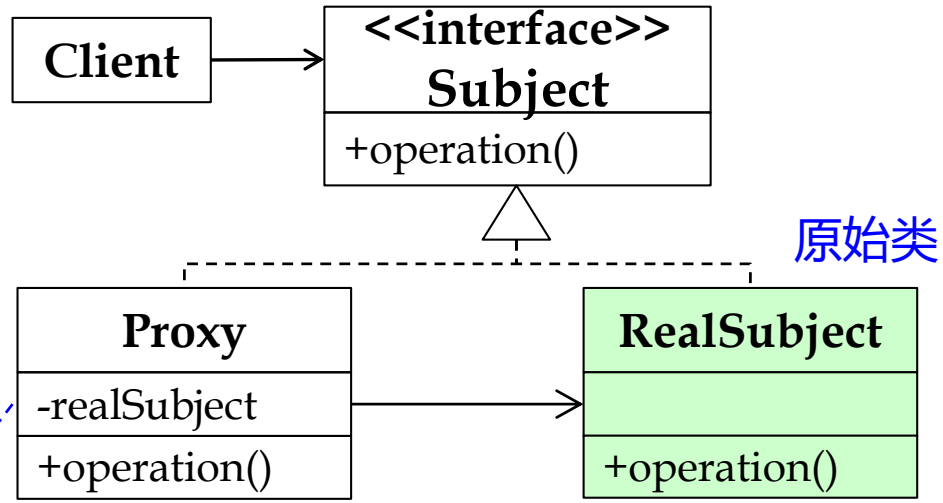
2.5 动态代理

- 所谓动态代理（Dynamic Proxy），就是不事先为每个原始类编写代理类，而是在运行的时候，动态地创建代理类，然后在系统中用代理类替换掉原始类

2.5 动态代理-①基于接口的动态代理（JDK动态代理）

- 如果原始类代码由我们开发维护，可以采用在内存中生成代理对象的方式。

前提条件：原始类实现了接口



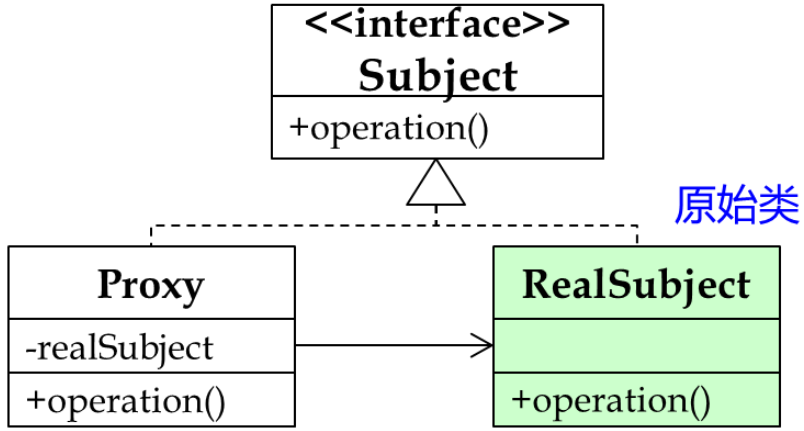
在运行的时候，动态地创建代理类，然后在系统中用代理类替换掉原始类。

2.5 动态代理-①基于接口的动态代理（JDK动态代理）

```
public interface IUserController {
    void login(String telephone, String password); //简化起见，将返回值设置为void
    void register(String telephone, String password); //简化起见，将返回值设置为void
    //其他的方法略...
}
```

```
public class UserController implements IUserController {  //必须实现接口
    //...省略其他属性和方法...
    @Override
    public void login(String telephone, String password) {//...省略login逻辑...
        System.out.println("执行登录操作");
    }

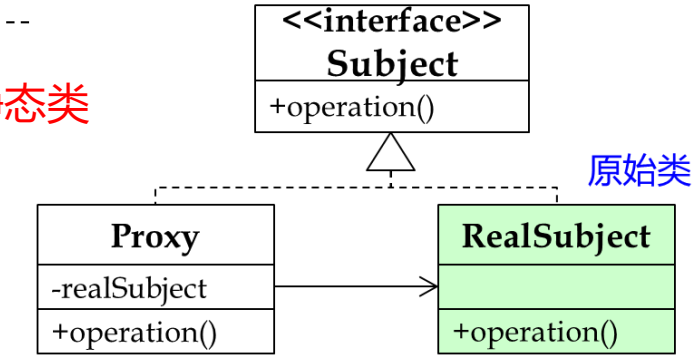
    @Override
    public void register(String telephone, String password) {
        //...省略 register 逻辑...
        System.out.println("执行注册操作");
    }
    // 其他代码略....
}
```



2.5 动态代理-①基于接口的动态代理（JDK动态代理）

```
public class Client {
    public static void main(String[] args) {
        //创建一个性能计数器对象
        MetricsCollector metricsCollector = new MetricsCollector();
        // 创建真实的业务对象
        IUserController userController = new UserController();
        // 使用动态代理创建代理对象
        IUserController proxyUserController = (IUserController) Proxy.newProxyInstance(
            // 参数1：类加载器，这里使用的是 IUserController 接口的类加载器
            IUserController.class.getClassLoader(),
            // 参数2：接口数组
            new Class[]{IUserController.class},
            // 参数3：InvocationHandler 用于处理所有接口方法的调用
            new InvocationHandler() {
                @Override
                public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
                    // 在方法执行前打印开始信息
                    String apiName = method.getName();
                    long startTimestamp = System.currentTimeMillis();
                    System.out.println(apiName + " 开始");
                    // 调用真实对象的方法
                    Object result = method.invoke(userController, args);
```

JDK提供的静态类



三个参数，
①获取接口的类加载器，
②接口数组和
③通用的处理器InvocationHandler 对象

横切逻辑

业务逻辑

被代理类的业务逻辑

2.5 动态代理- ①基于接口的动态代理（JDK动态代理）

横切逻辑

```
                // 在方法执行后打印结束信息以及耗时
                long endTimeStamp = System.currentTimeMillis();
                long responseTime = endTimeStamp - startTimestamp;
                System.out.println(apiName + " 结束，耗时：" + responseTime + "ms");
                RequestInfo requestInfo = new RequestInfo(apiName, responseTime, startTimestamp);
                metricsCollector.recordRequest(requestInfo);
                return result;
            }
        }
    ); // end of new InvocationHandler

    // 调用代理对象的方法
    proxyUserController.login("12","23");
    proxyUserController.register("4545454","4545");
  }
}
```

```
D:\tools\jdk-17.0.12\bin\java.exe "-ja
login 开始
执行登录操作
login 结束，耗时：2ms
register 开始
执行注册操作
register 结束，耗时：0ms
```

我们发现，能够在内存中生成动态代理类。而且能够为原始类前后添加横切逻辑。

问题是：每一次调用 UserController 或者其他需要统计信息的类，都需要写一堆代码。

2.5 动态代理- ①基于接口的动态代理（JDK动态代理）

解决方案：将这些代码抽取成一个类。

```
public class ProxyObjectCreator {
    // 注入信息采集器
    private MetricsCollector metricsCollector;
    // 在构造函数中初始化
    public ProxyObjectCreator() {
        metricsCollector = new MetricsCollector();
    }
    // 创建代理对象
    public Object createProxy(Object proxiedObject) {
        Class<?>[] interfaces = proxiedObject.getClass().getInterfaces(); // 原始类实现的接口
        DynamicProxyHandler handler = new DynamicProxyHandler(proxiedObject); //执行的方法逻辑
        // 调用Java 的Proxy类创建代理对象
        return Proxy.newProxyInstance(
                        proxiedObject.getClass().getClassLoader(),
                        interfaces,
                        handler);
    }

    // 接下一页
```

原始类的对象

做成内部类

2.5 动态代理- ①基于接口的动态代理（JDK动态代理）

```
    private class DynamicProxyHandler implements InvocationHandler { // 私有的内部类
        private Object proxiedObject;
        public DynamicProxyHandler(Object proxiedObject) {
            this.proxiedObject = proxiedObject;
        }
        @Override
        public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
            long startTimestamp = System.currentTimeMillis();
            String apiName = proxiedObject.getClass().getName() + ":" + method.getName();
            System.out.println("apiName: " + apiName);

            Object result = method.invoke(proxiedObject, args);

            long endTimeStamp = System.currentTimeMillis();
            long responseTime = endTimeStamp - startTimestamp;
            System.out.println( apiName + ", 执行时间: " + responseTime + "ms");
            RequestInfo requestInfo = new RequestInfo(apiName, responseTime, startTimestamp);
            metricsCollector.recordRequest(requestInfo);
            return result;
        }
    }
}
```

横切逻辑

业务逻辑

横切逻辑

2.5 动态代理- ①基于接口的动态代理（JDK动态代理）

- ProxyObjectCreator 使用举例

```
public class Client {
    public static void main(String[] args) {
        ProxyObjectCreator proxyObjectCreator = new ProxyObjectCreator();
        IUserController proxy = (IUserController) proxyObjectCreator.createProxy(new UserController());
        proxy.login("admin", "admin");
        proxy.register("hello", "world");
    }
}
```

```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.4.1\lib\idea_rt
调用接口: proxy.dy.UserController:login
执行登录操作
接口运行时间： proxy.dy.UserController:login, 执行时间: 12ms
调用接口: proxy.dy.UserController:register
执行注册操作
接口运行时间： proxy.dy.UserController:register, 执行时间: 0ms
```

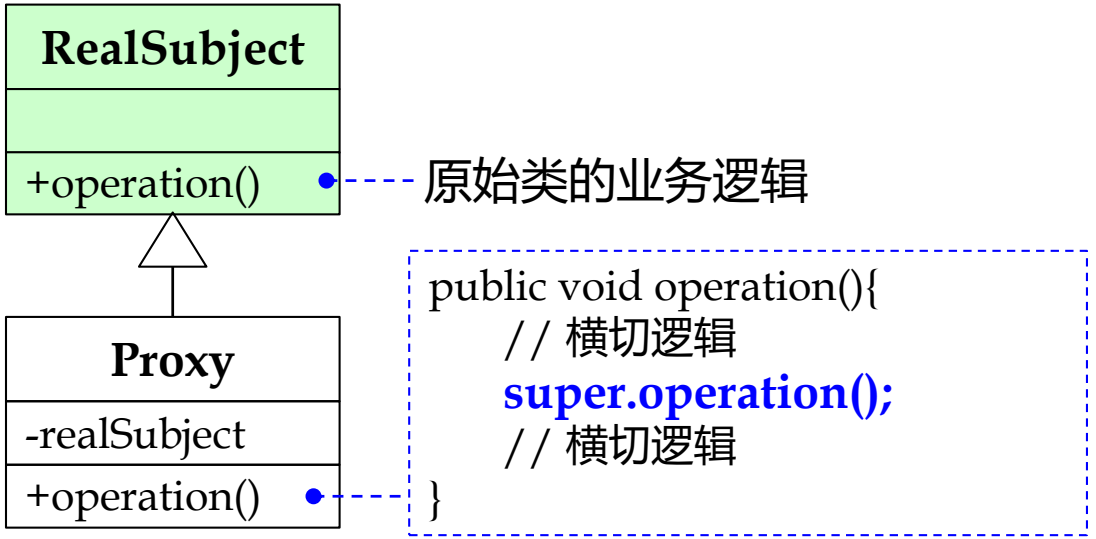
2.5 动态代理- ②基于继承的动态代理（CGLIB代理）

- 目标对象只是一个单独的对象，并没有实现任何的接口，这个时候可使用目标对象的子类来实现动态代理

基于继承实现代理模式

在子类中覆写父类的方法，添加横切逻辑。

在编程过程中，用子类替换父类



CGLIB（Code Generation Library）是一个广泛用于Java中的动态代理库，通常与 Spring 等框架一起使用。它的主要特点是不需要目标对象实现接口，而是通过生成目标对象的子类来实现代理

cglib github 官方地址：https://github.com/cglib/cglib

2.5 动态代理- ②基于继承的动态代理（CGLIB代理）

- 在Java 的Maven工程中，添加依赖：

```
<dependency>
    <groupId>cglib</groupId>
    <artifactId>cglib</artifactId>
    <version>3.3.0</version>
</dependency>
```

- 原始类如下（省略了返回值，省略了参数）：

```
public class UserController {
    public void login() {
        System.out.println("登录逻辑....");
    }

    public void register() {
        System.out.println("注册逻辑....");
    }
}
```

2.5 动态代理- ②基于继承的动态代理（CGLIB代理）

```
public class MyMethodInterceptor implements MethodInterceptor {
    Object target; //target代表被代理类的实例对象

    public MyMethodInterceptor(Object target) {
        this.target = target;
    }

    @Override
    public Object intercept(Object o, Method method, Object[] args, MethodProxy methodProxy)
                                                                        throws Throwable {
        // 实现增强代码的地方，这里就用一个简单的打印语句代替一下了
        System.out.println("增强的代码：前置通知");

        // 执行目标方法，这里是直接调用父类方法(就是直接调用被代理类的方法)，相当于执行super.方法()
        Object returnValue = methodProxy.invoke(target, args);

        System.out.println("增强的代码：后置通知");

        return returnValue;
    }
}
```

横切逻辑

业务逻辑

横切逻辑

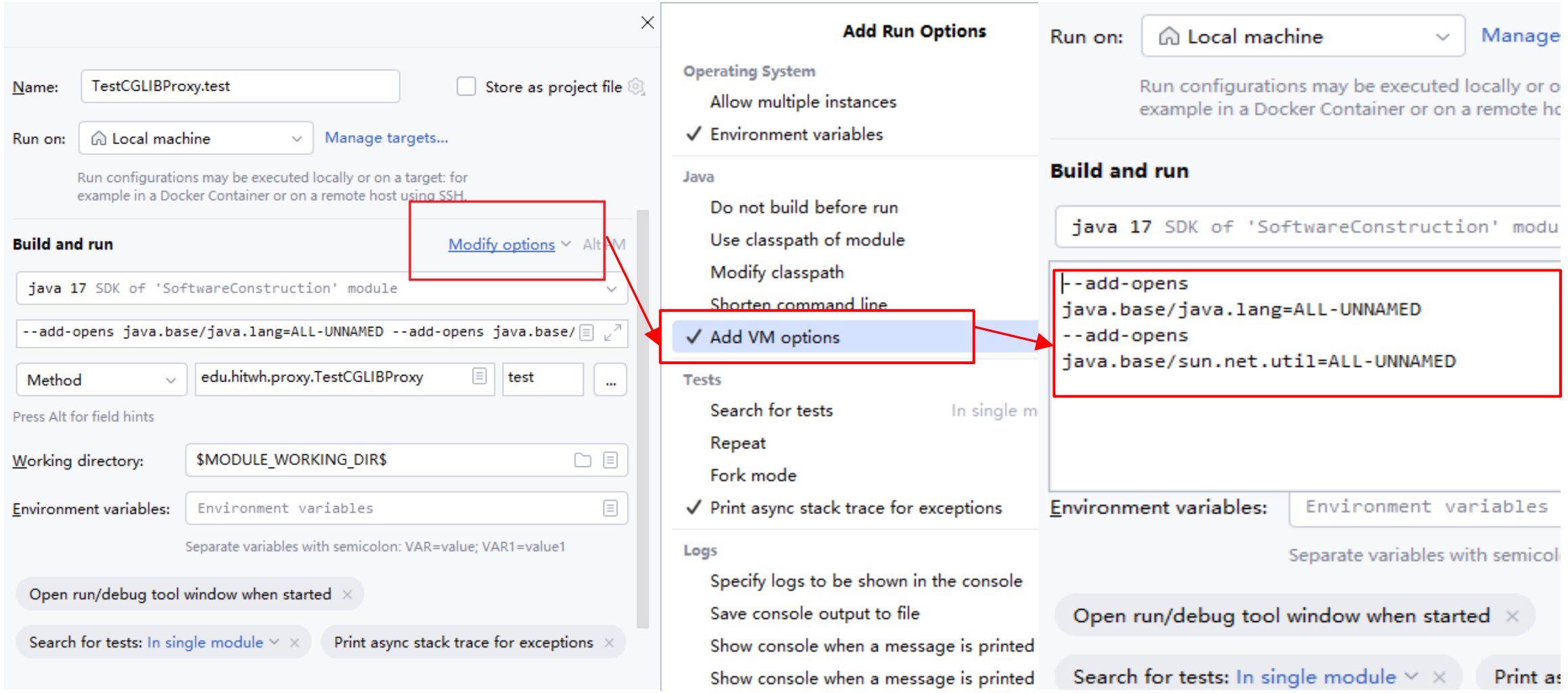
2.5 动态代理- ②基于继承的动态代理（CGLIB代理）

```
public class TestCGLIBProxy {
    @Test
    public void test(){
        //创建代理工厂类实例
         Enhancer enhancer = new Enhancer();
        //设置被代理的类
         enhancer.setSuperclass(UserController.class);
        //设置处理类
         enhancer.setCallback(new MyMethodInterceptor(new UserController()));
        //生成的代理类，这个代理类是继承自 UserController 的
         UserController userController = (UserController) enhancer.create();
        //打印结果
        System.out.println(userController.getClass());
        userController.login(); //通过代理对象调用方法
    }
}
```

```
✓ 1 test passed  1 test total, 143 ms
D:\tools\jdk-17.0.9\bin\java.exe ...
class edu.hitwh.proxy.UserController$$EnhancerByCG
增强的代码：前置通知
登录逻辑....
增强的代码：后置通知
```

CGLIB 与 JDK8 以上不兼容，依赖失效。运行时，需要加 JVM 参数，选择开启不被允许的反射功能：
--add-opens java.base/java.lang=ALL-UNNAMED　(注意，前面是两个短横)
--add-opens java.base/sun.net.util=ALL-UNNAMED

2.5 动态代理- ②基于继承的动态代理（CGLIB代理）



2.6 代理模式的常见应用场景

序号	场景	说明
1	业务系统的非功能性需求开发	监控、统计、鉴权、限流、事务、幂等、日志。 将这些附加功能与业务功能解耦，放到代理类中统一处理，让程序员只需要关注业务方面的开发。 Spring AOP 底层的实现原理就是基于动态代理
2	代理模式在 RPC的应用	RPC 框架也可以看作一种代理模式 通过远程代理，将网络通信、数据编解码等细节隐藏起来。
3	代理模式在缓存中的应用	实时查询可以看做是正常的业务逻辑，缓存查询做成 AOP 切面逻辑

2.7 代理模式小结

特性	静态代理 (Static Proxy)	动态代理 (Dynamic Proxy)
创建时机	编译时 (代理类 .java 文件已存在)	运行时 (代理类在内存中动态生成)
代码量	大，一个代理类对应一个目标类	小，一个 Handler 可服务于多个目标类
灵活性	低，接口变更需要同步修改代理类	高，接口变更无需修改代理逻辑
耦合度	代理类与目标类紧耦合	代理逻辑与目标类松耦合
适用场景	代理逻辑简单且固定，代理类数量少的场景	框架开发，如 Spring AOP、RPC 等需要对大量类进行通用增强的场景

	JDK 动态代理	动态代理 (Dynamic Proxy)
优点	实现简单，原生支持，依赖较少 适合轻量级、简单场景。更稳定	CGLIB 动态代理
缺点	性能在高频调用时明显不如 CGLIB	使用更复杂，可能引入额外依赖需要为目标类生成子类，因此无法代理 final 类或 final 方法

Spring AOP 是用哪个代理方式的？如果目标对象实现了接口，默认使用 JDK 动态代理；如果目标对象没有接口，使用 CGLIB 动态代理（可以强制使用CGLIB：在配置文件中设置 proxy-target-class=true）

3. 装饰模式

Decorator Design Pattern

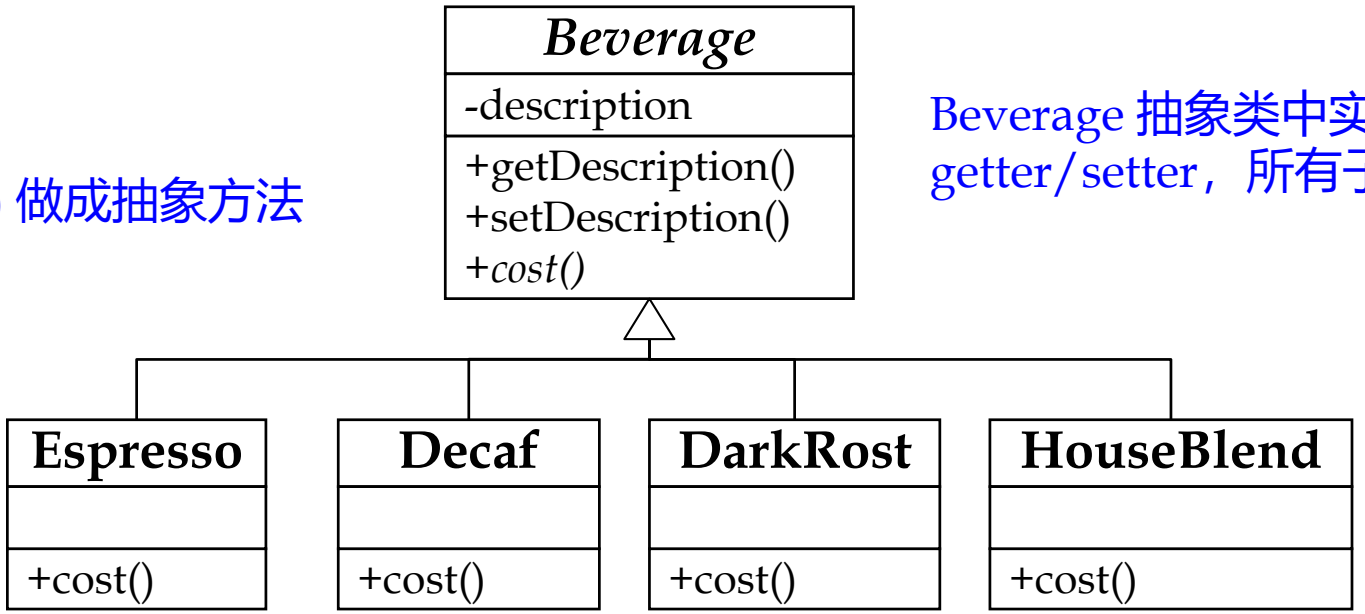
3.1 引例：需求描述

- Starbuzz Coffee 咖啡店除 Espresso，Decaf，DarkRost 等常规咖啡品种外，需要不断扩展新的咖啡品种；为了适应业务，急需更新计费系统。在更新后的计费系统中，咖啡和调料都需要收费
- 目前亟需解决的问题：
  - 咖啡的品种不断增加；
  - 调料的品种逐步更新；
  - 每位顾客点的咖啡和调料不尽相同
- Starbuzz Coffee 希望尽快找到解决方案，实现能够满足其需求的计费系统。

3.1 引例：需求分析

- Starbuzz Coffee 卖的是饮料，无论何种咖啡，都是饮料的一种，因此，应该将各种咖啡抽象成一个超类。
- 每种咖啡都应该有个描述 description, 保存该饮料的具体说明信息，以便顾客了解这种饮料

在超类中 cost() 做成抽象方法



Beverage 抽象类中实现description的getter/setter，所有子类自动继承即可

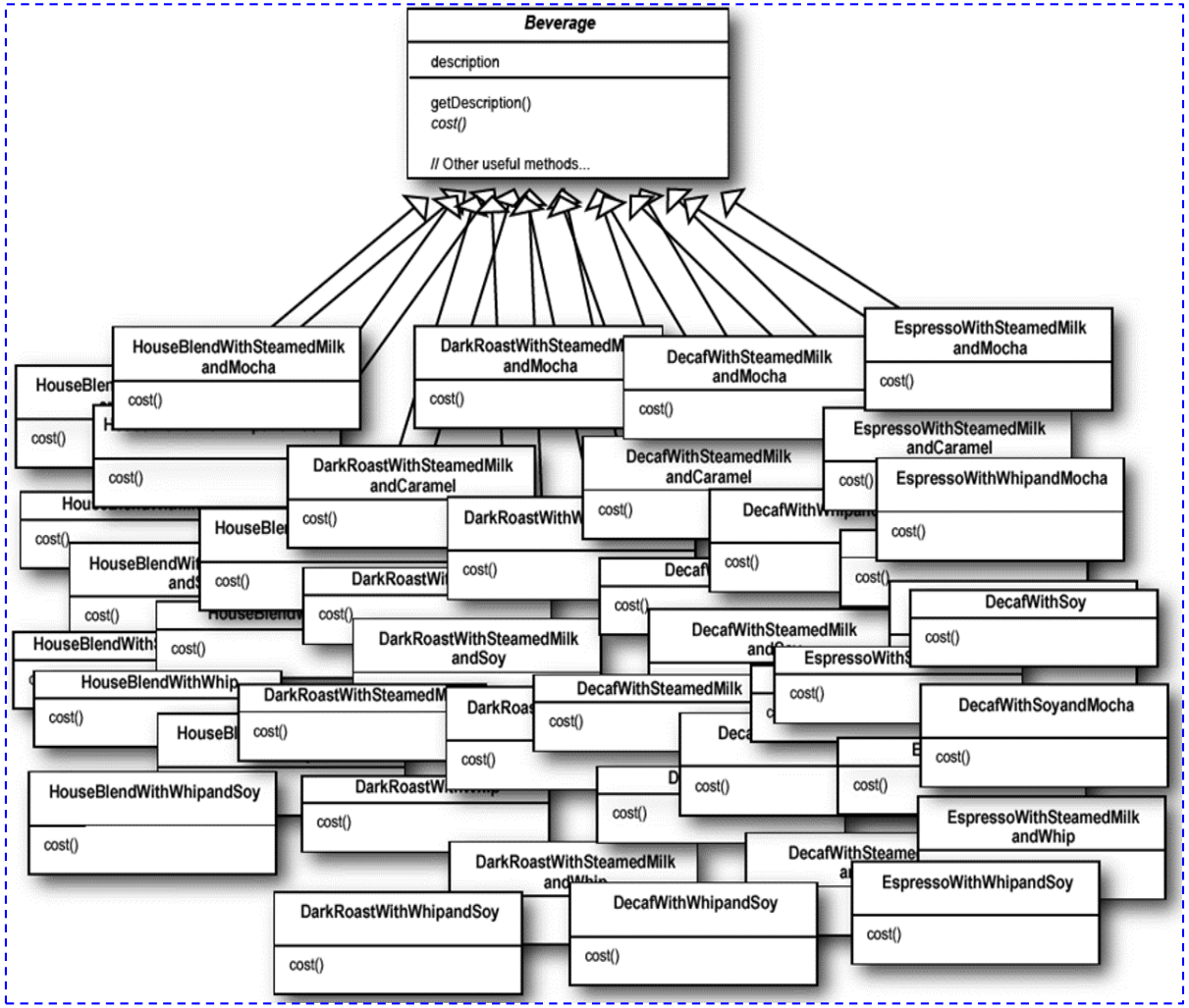
每种咖啡应该有计算价格的方法 cost()，以便用户结算，且每种咖啡都有自己的计算方式。

3.1 引例：需求分析

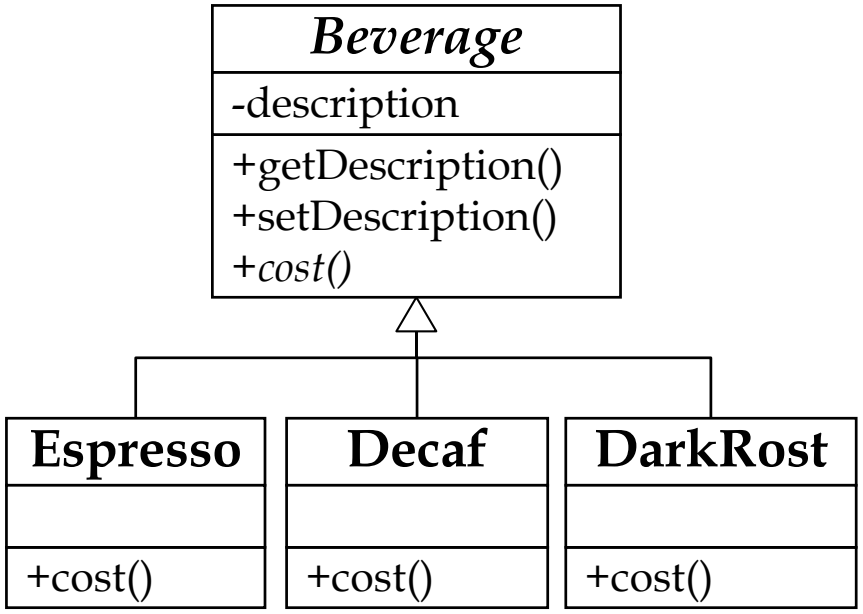
- 除了咖啡类，还需要对调料进行处理
  - Steamed Milk（蒸过的牛奶）
  - Soy（豆酱）
  - Mocha（糊状巧克力）
  - Whip（生牛奶 ）
  - 其他调料
- 所有这些额外调料都是收费的，在构建计费系统时候必须考虑到这些调料。顾客对调料的需求有多种情况：
  - 可能不加调料；可能加一份某种调料；可能加多份某种调料；可能加多份、多种调料
- 如何构建这套系统，使其能够适应咖啡和调料品种的增加，并且咖啡和调料都能计费？

3.2 设计方案讨论：排列组合

- 是否可以采用排列组合思想，将咖啡各子类 与 调料进行排列组合，形成不同的类?
  - 加 steamed milk 的 Espresso，Decaf，DarkRost，HouseBlend 等咖啡子类；
  - 加 Soy 的 Espresso，Decaf，DarkRost，HouseBlend
  - …
- 这种穷举“咖啡和调料形成的组合”的设计方案，必然形成庞大的类系，造成类的大爆炸
- 结论：显然不合适，否决该方案。



3.2 设计方案讨论：利用继承



能否将调料的计算放到超类中?

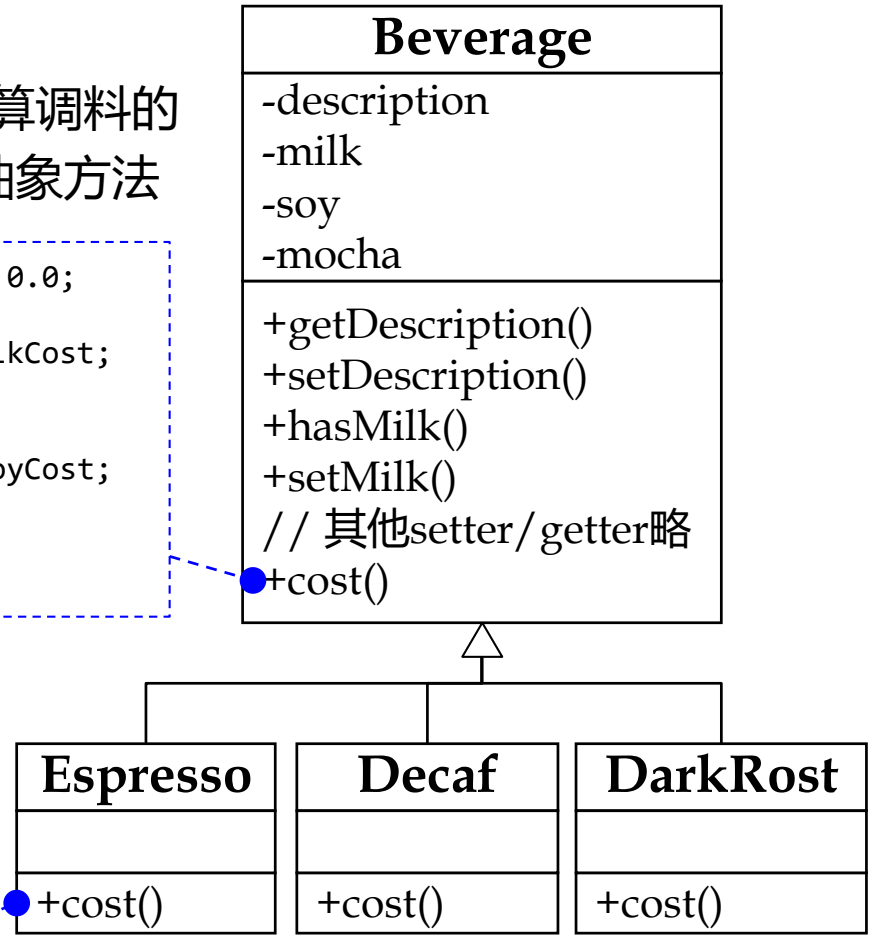
子类的 cost() 这样写：

1.2 + super.cost()

咖啡的价格　　调料的价格

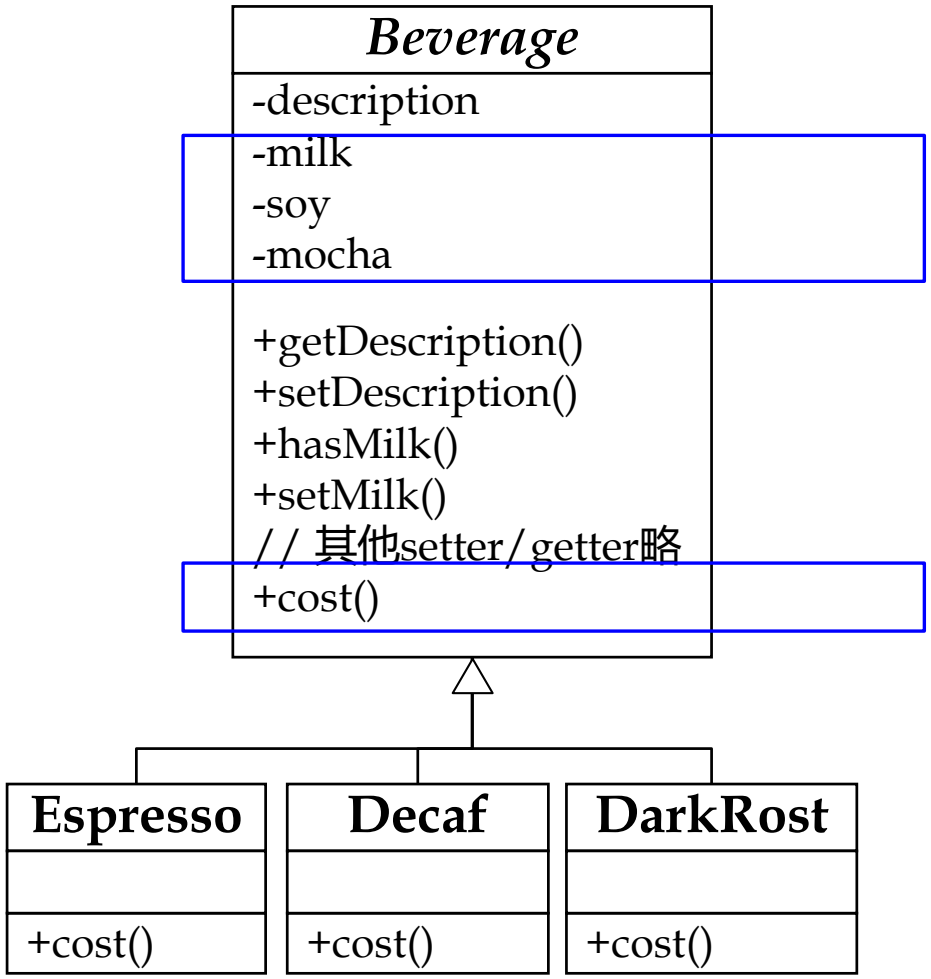
超类的cost()负责计算调料的价格，因此不再是抽象方法

```
double condimentCost = 0.0;
if (hasMilk()) {
   condimentCost += milkCost;
}
if (hasSoy()) {
    condimentCost += soyCost;
}
//…
return condimentCost;
```




```
return 1.99 + super.cost();
```

3.2 设计方案讨论：利用继承



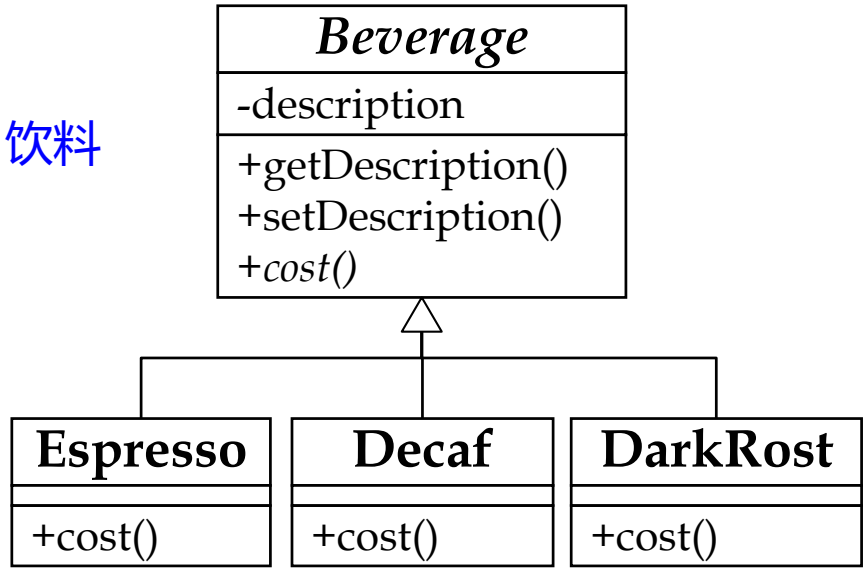
调料价格变化需要改变现有代码，加入新的调料时，必须给超类添加新的方法并且改变 cost 方法的实现

如果用户想要双份巧克力，怎么办?

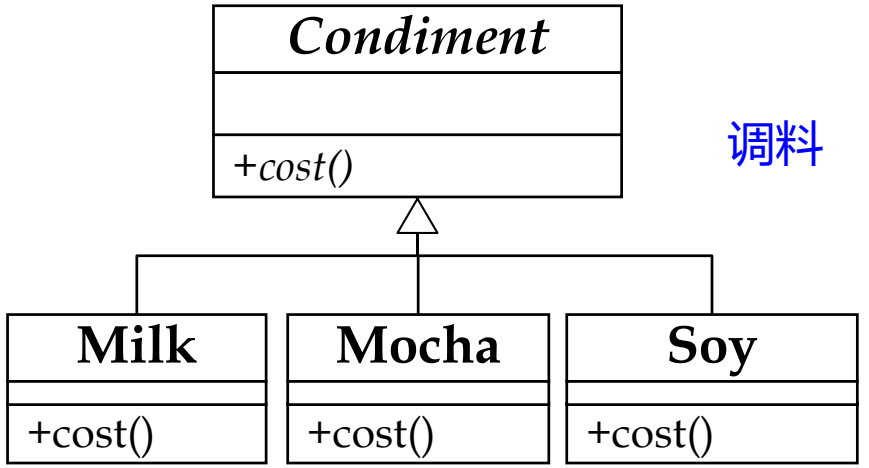
结论：
该方案避免了类爆炸，实现了咖啡与调料分别计算。
但是，满足不了需求的变化，不符合 OCP，因此该方案不可行。

3.2 设计方案讨论：分离-组合的思想

饮料



调料



将调料类和饮料类分离，各自计算自己的价格

如何将这两套类系组合在一起？需要解决如下问题：

① 能够将不同的咖啡+调料的价格计算出来；

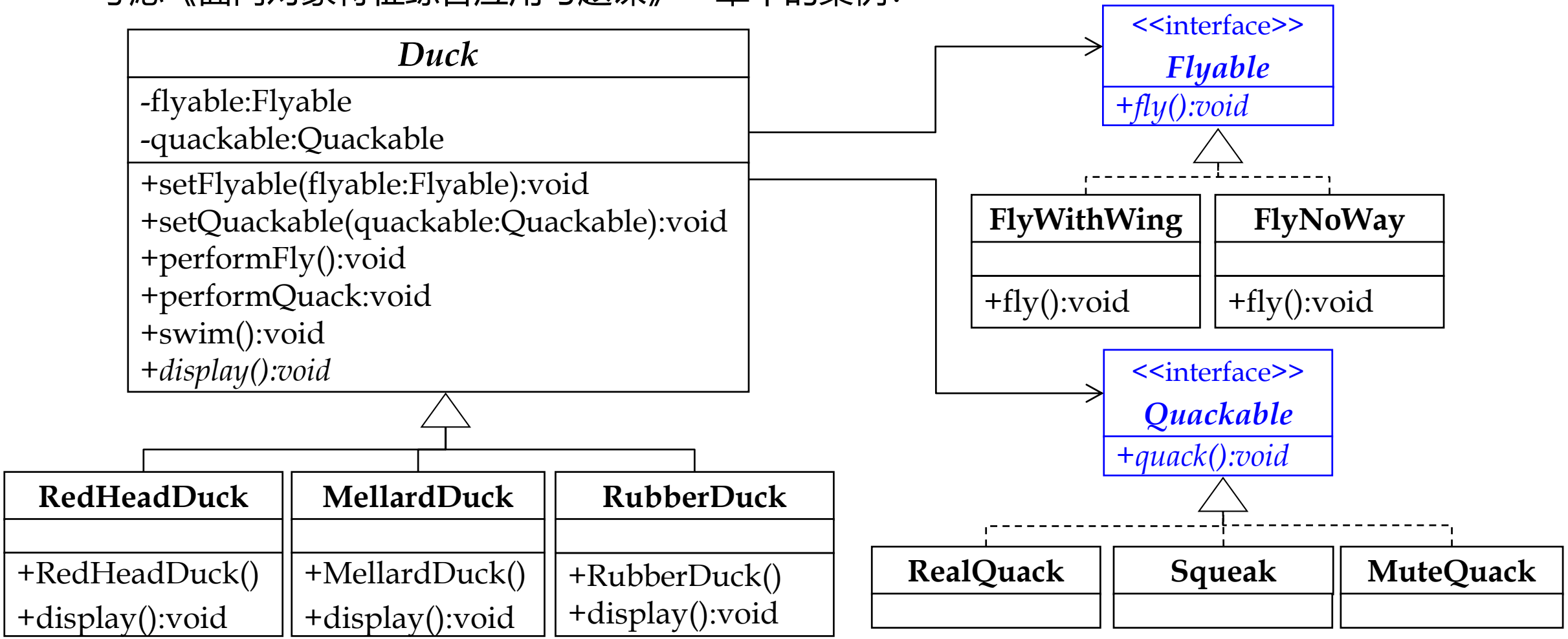
② 并且满足咖啡，调料两个维度都能够变化；

③ 咖啡中可以加任意种类、任意份调料；

④ 尽量满足 OCP

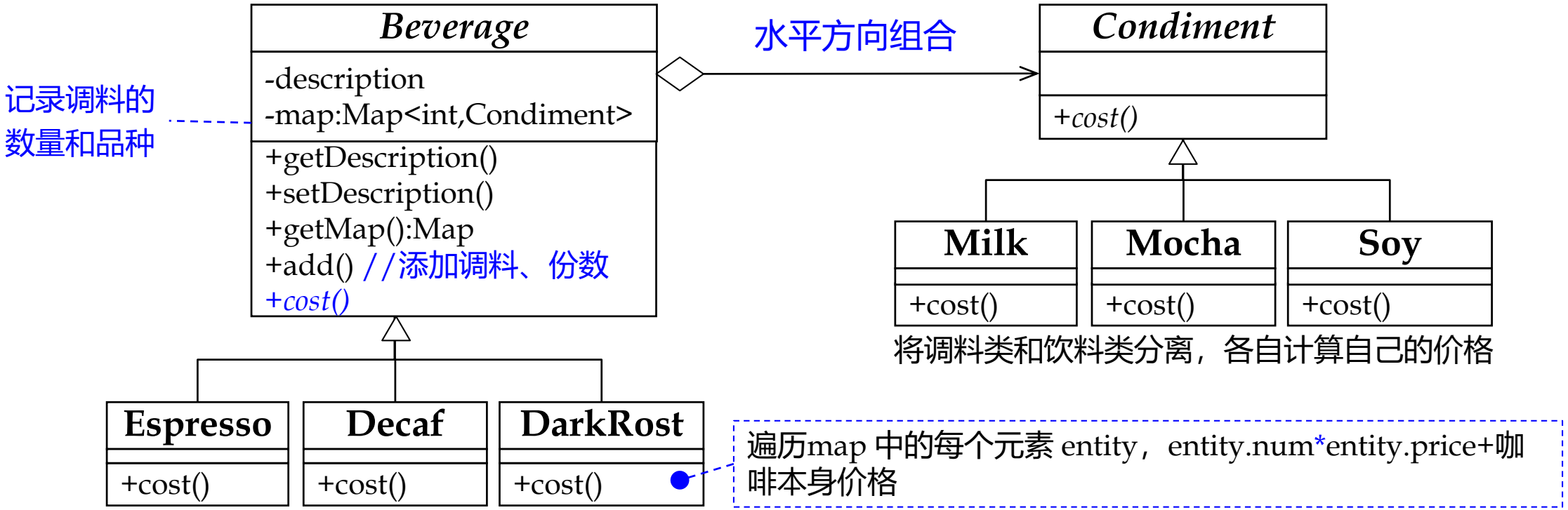
3.2 设计方案讨论：分离-组合的思想

- 考虑《面向对象特征综合应用习题课》一章中的案例：



3.2 设计方案讨论：分离-组合的思想

- 很容易想到，将调料类组合进咖啡超类中。考虑到，可能加多份多种调料，因此应该采用一个Map进行存储调料的信息。HashMap<int,Condiment> //数量，调料对象

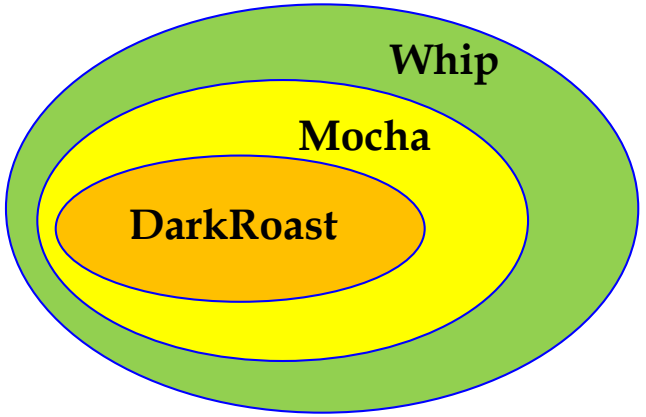


在客户端中，设置某种咖啡对象需要的调料和分数，加到 map 对象即可。搞定。

这种思想实际上是“桥接模式”的设计思想，下一节讲述。本节我们采用另一种思路，讨论是否可行：

3.2 设计方案讨论：分离-组合的思想

- 一杯 DarkRoast咖啡中加入 Mocha 和 Whip，可以看成三步：



- ① 首先产生一杯咖啡 DarkRoast
- ② 其次在DarkRoast 开发中加一份 Mocha，形成一份混合物，即：DarkRoast + Mocha
- ③ 最后在混合物中再加一份Whip，形成新的混合物，即：DarkRoast + mocha + Whip

混合后饮料 = (((DarkRost) + Mocha) + whip)

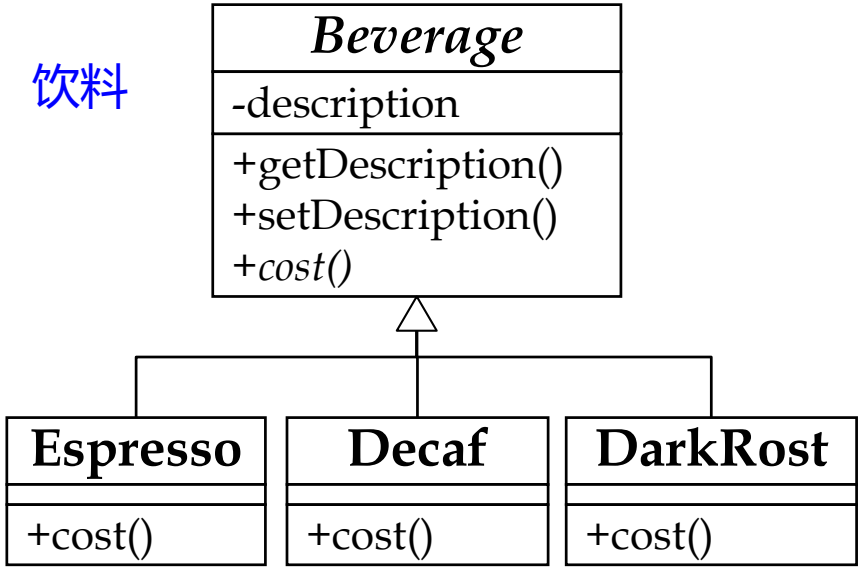
混合后饮料价格 = 混合后饮料价格 + 调料类价格

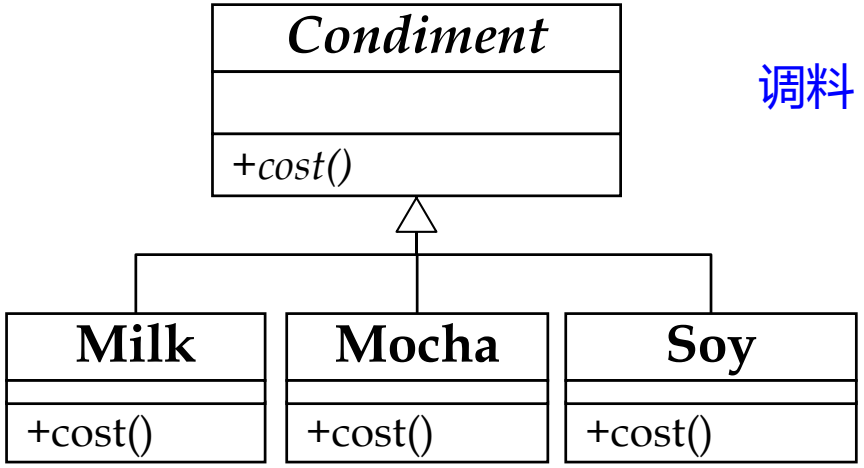
可以发现，以上等式形成一种递归关系

3.2 设计方案讨论：分离-组合的思想

- 如何在如下类图中表达此种递归关系？

饮料





调料

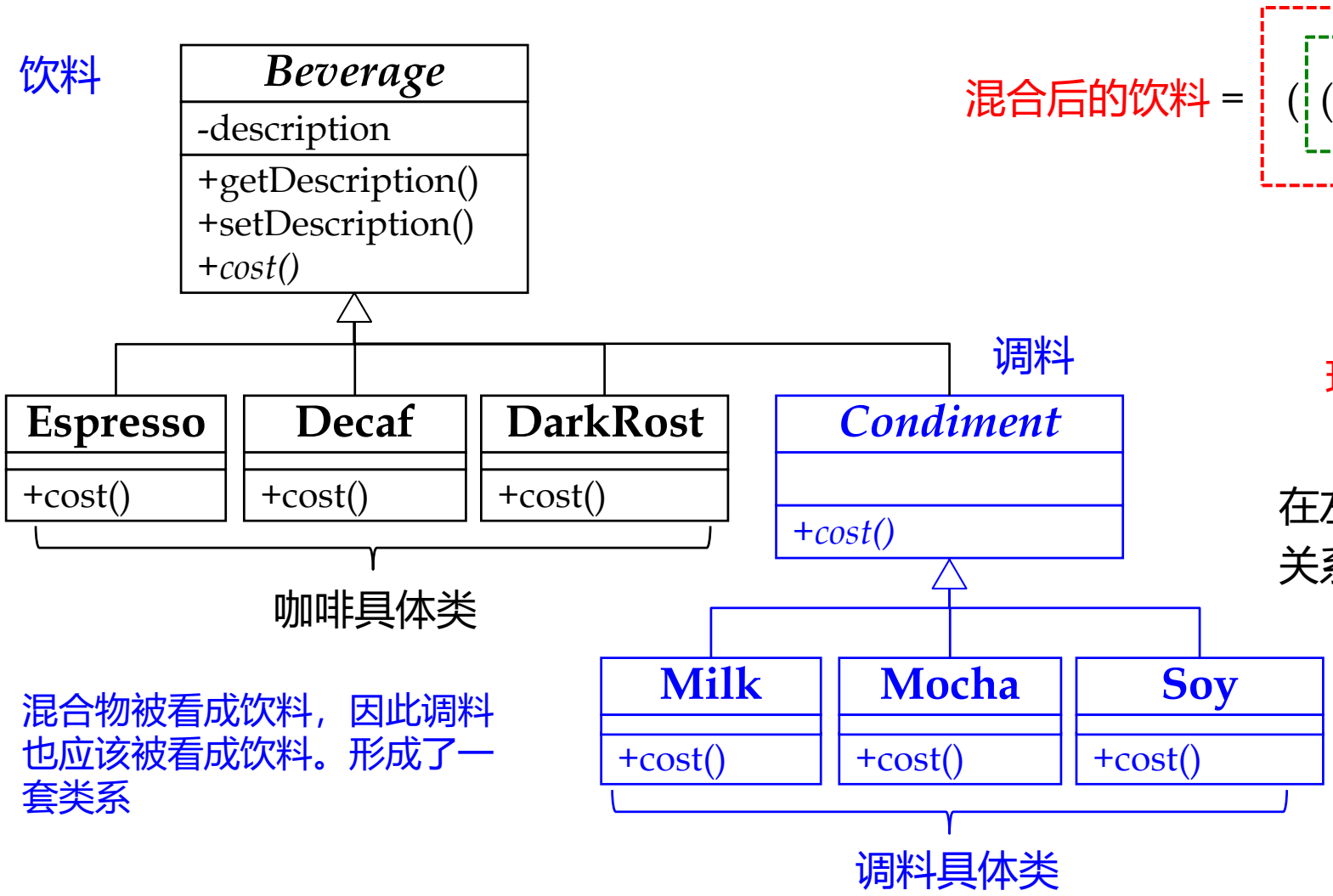
混合后饮料价格 = 混合后饮料价格 + 调料类价格

可以发现，以上等式形成一种递归关系

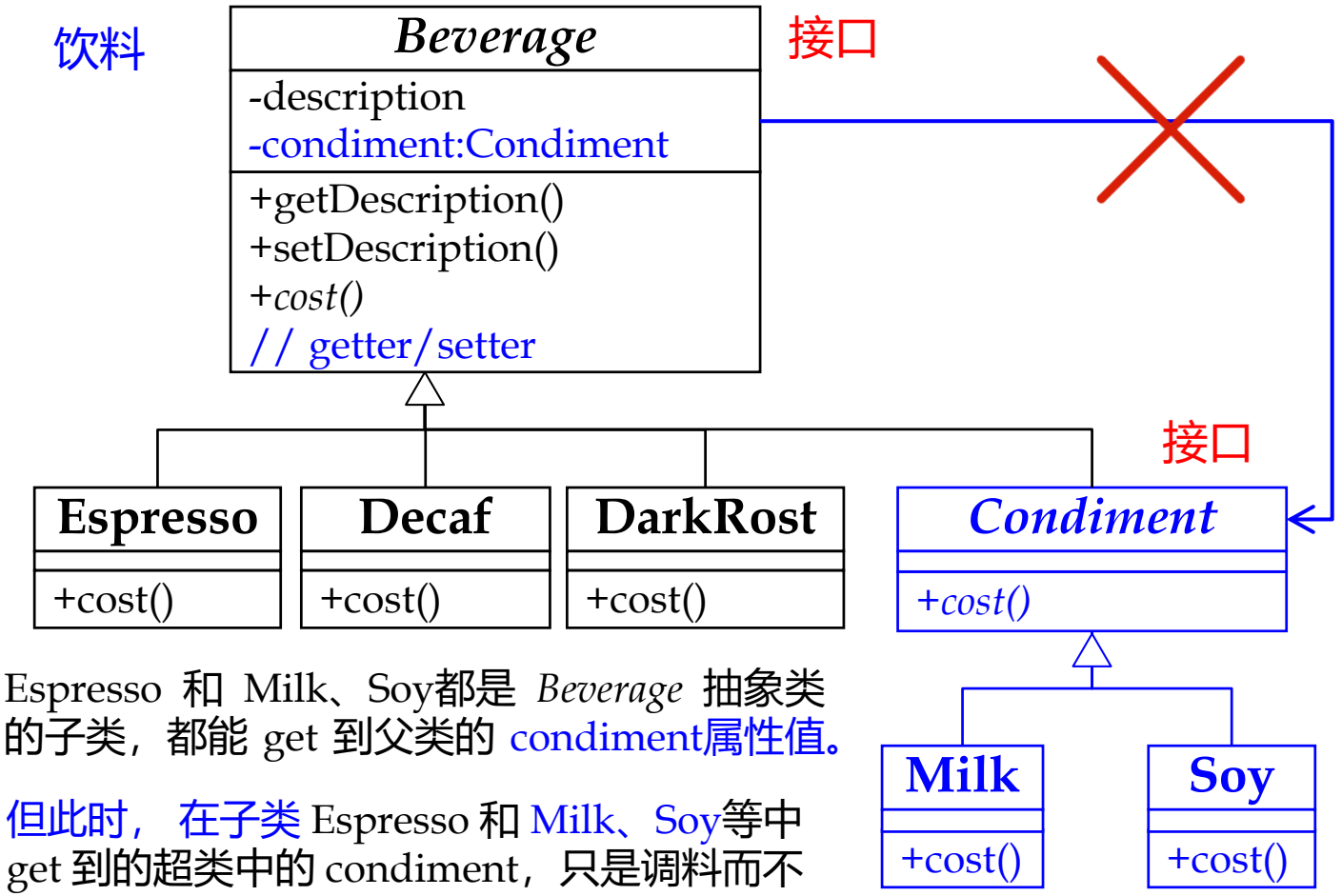
混合物被看成饮料，因此调料也应该被看成饮料。

否则混合物的概念就不成立了。

3.2 设计方案讨论：分离-组合的思想



3.2 设计方案讨论：分离-组合的思想



Espresso 和 Milk、Soy都是 *Beverage* 抽象类的子类，都能 get 到父类的 condiment属性值。

但此时， 在子类 Espresso 和 Milk、Soy等中 get 到的超类中的 condiment，只是调料而不是混合物

观点①：应该通过组合方式，Beverage 中有调料对象 condiment

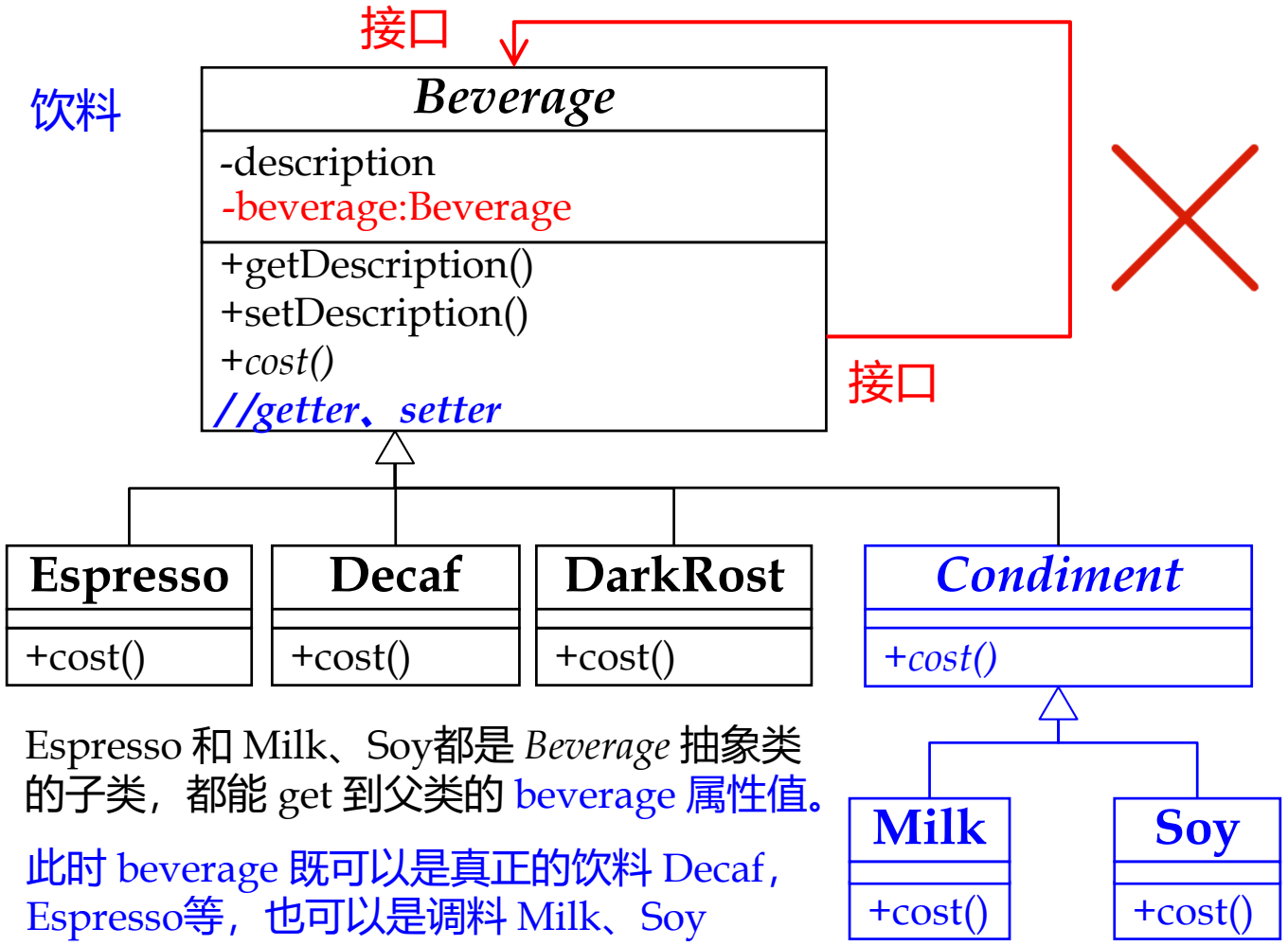
面向接口编程，应该是接口之间的关系 Beverage has Condiment

此时 Beverage 中的属性 condiment 对象只能指向 Condiment 的子类；

语义上：咖啡有调料；调料有调料。

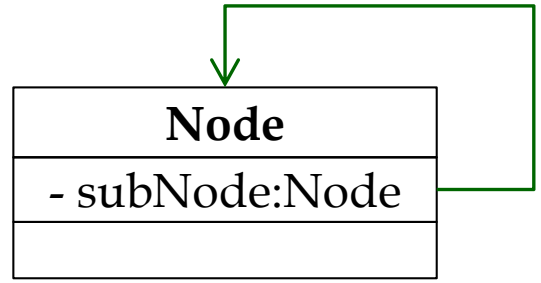
无法表达混合物，也就无法形成递归。

3.2 设计方案讨论：分离-组合的思想



Espresso 和 Milk、Soy都是 *Beverage* 抽象类的子类，都能 get 到父类的 beverage 属性值。

此时 beverage 既可以是真正的饮料 Decaf，Espresso等，也可以是调料 Milk、Soy



自关联：自己包含自己

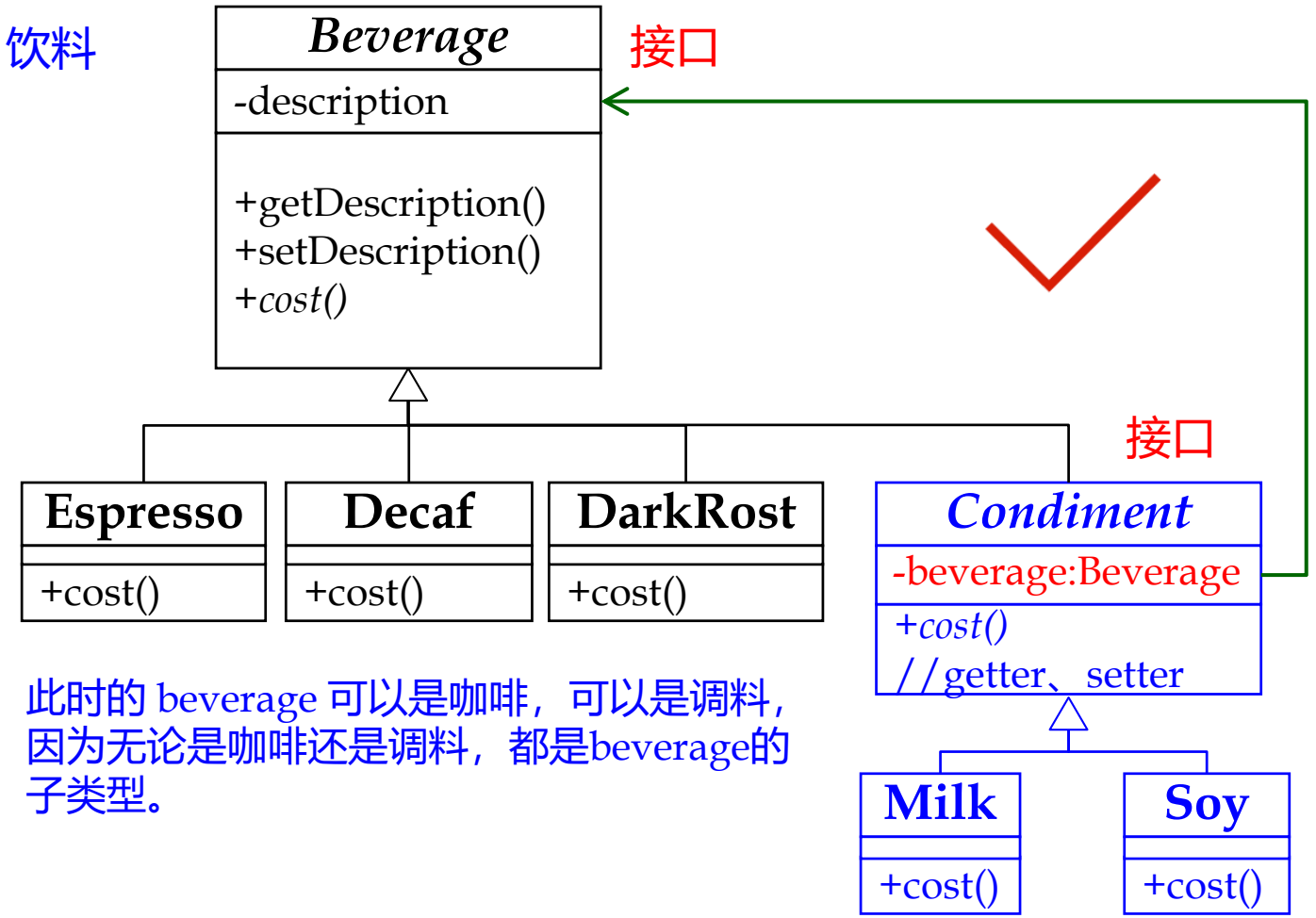
观点②：干脆让Beverage自关联

显然，此时能够形成递归的概念。

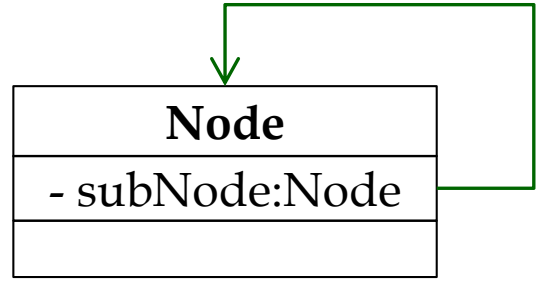
Beverage 代表本类系中一切，因此在子类中，咖啡 has 调料；调料 has 调料；咖啡 has 咖啡；

可以出现咖啡和咖啡混合。显然将需求的范围扩大了。（咖啡中不能再has咖啡）

3.2 设计方案讨论：分离-组合的思想



此时的 beverage 可以是咖啡，可以是调料，因为无论是咖啡还是调料，都是beverage的子类型。



自关联：自己包含自己

观点③：调料对象中有饮料超类对象

面向接口编程，应该是接口之间的关系

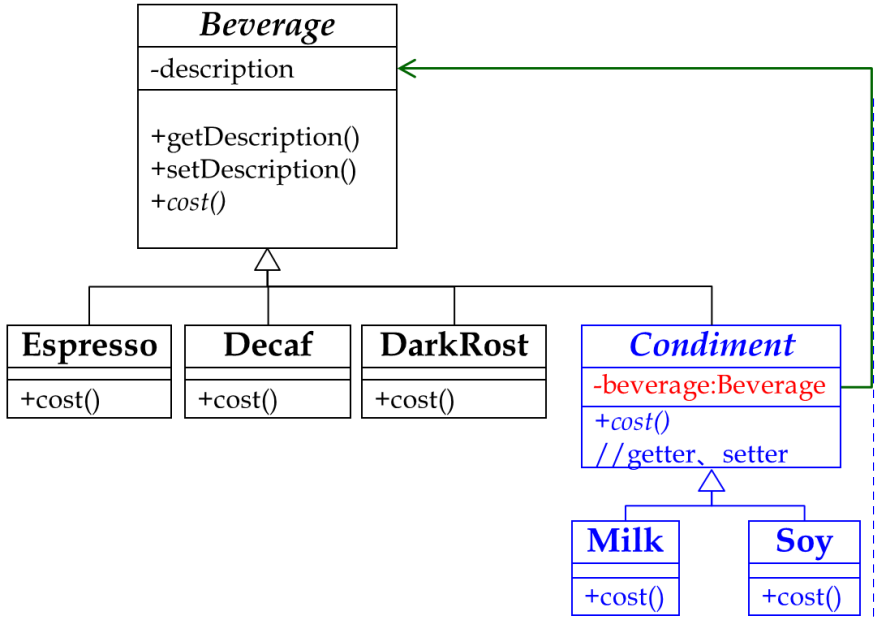
Condiment has Beverage

此时，调料 has 调料；调料 has 咖啡；也可以调料 has 混合物。

但是限制了“咖啡不能 has 别的对象”，从而解决了“咖啡 has 咖啡” 这种包含范围过大的问题。

3.2 设计方案讨论：分离-组合的思想

```
public abstract class Beverage {
    private String description = "Unknown Beverage";
    public void setDescription(String description) {
        this.description = description;
    }
    public String getDescription() {
        return description;
    }
    public abstract double cost();
}
```

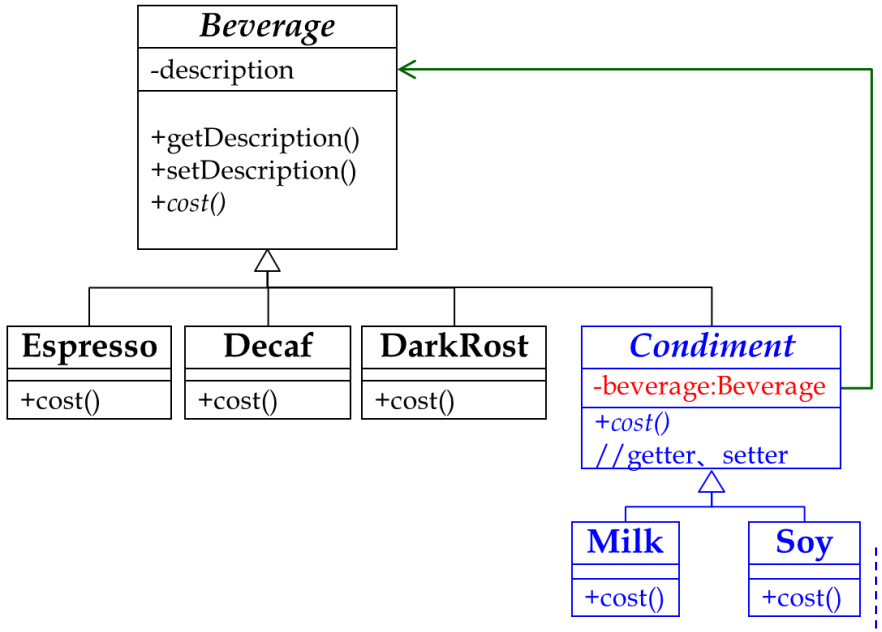



```
public abstract class Condiment extends Beverage {   //未实现超类中的cost()抽象方法，故依然是抽象的
    private Beverage beverage;
    public Beverage getBeverage() {
        return beverage;
    }
    public void setBeverage(Beverage beverage) {
        this.beverage = beverage;
    }
}
```

3.2 设计方案讨论：分离-组合的思想

```
public class DarkRoast extends Beverage {
    public DarkRoast() {
        this.setDescription("Dark Roast Coffee");
    }
    public double cost() {
        return 1.0;
    }
}
```

```
public class Whip extends Condiment {
    public Whip(Beverage beverage) {
        this.setBeverage(beverage);
    }
    public String getDescription() {
        return this.getBeverage().getDescription() + ", Whip";
    }
    public double cost() {
        return 0.40 + this.getBeverage().cost();    // 自身价格+混合物的价格
    }
}
```

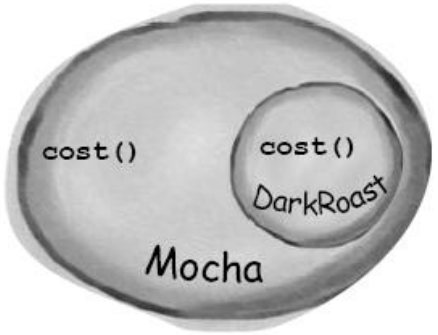


3.2 设计方案讨论：分离-组合的思想

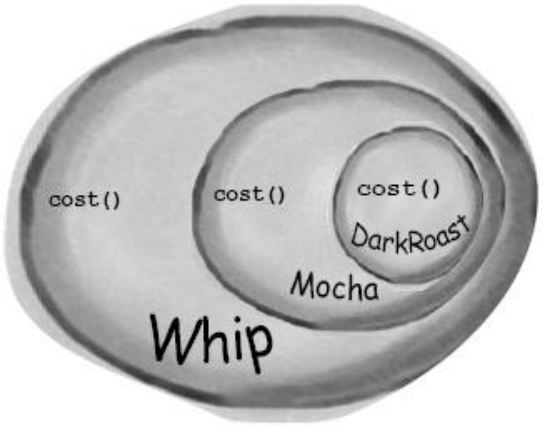
- 测试：带有巧克力(mocha)和焦糖(Whip)的 Dark Roast 咖啡计费：



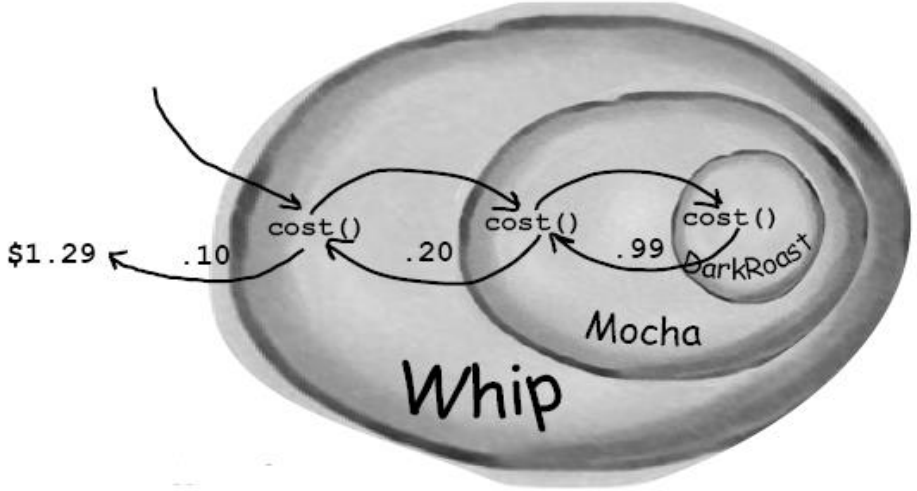
①：生成 DarkRost 咖啡对象



②：生成Mocha对象，包装 DarkRost 对象



③：生成Whip对象，包装加了 Mocha 的 DarkRoast 对象



④：从最外层 Whip 的方法cost()开始计算，逐步计算到最里层的咖啡类（原始类），最后返回整个混合物的价格

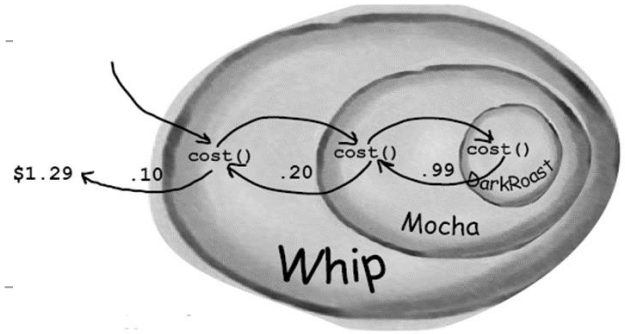
```java
public class Client {
    public static void main(String[] args) {
        Beverage beverage = new DarkRoast();

        beverage = new Mocha(beverage);

        beverage = new Whip(beverage);

        System.out.println(beverage.getDescription() + " $" + beverage.cost());
    }
}
```

递归调用每个对象的cost()



cost()

Whip+Mocha+DarkRost 混合物

Mocha+DarkRost 混合物

DarkRost 混合物

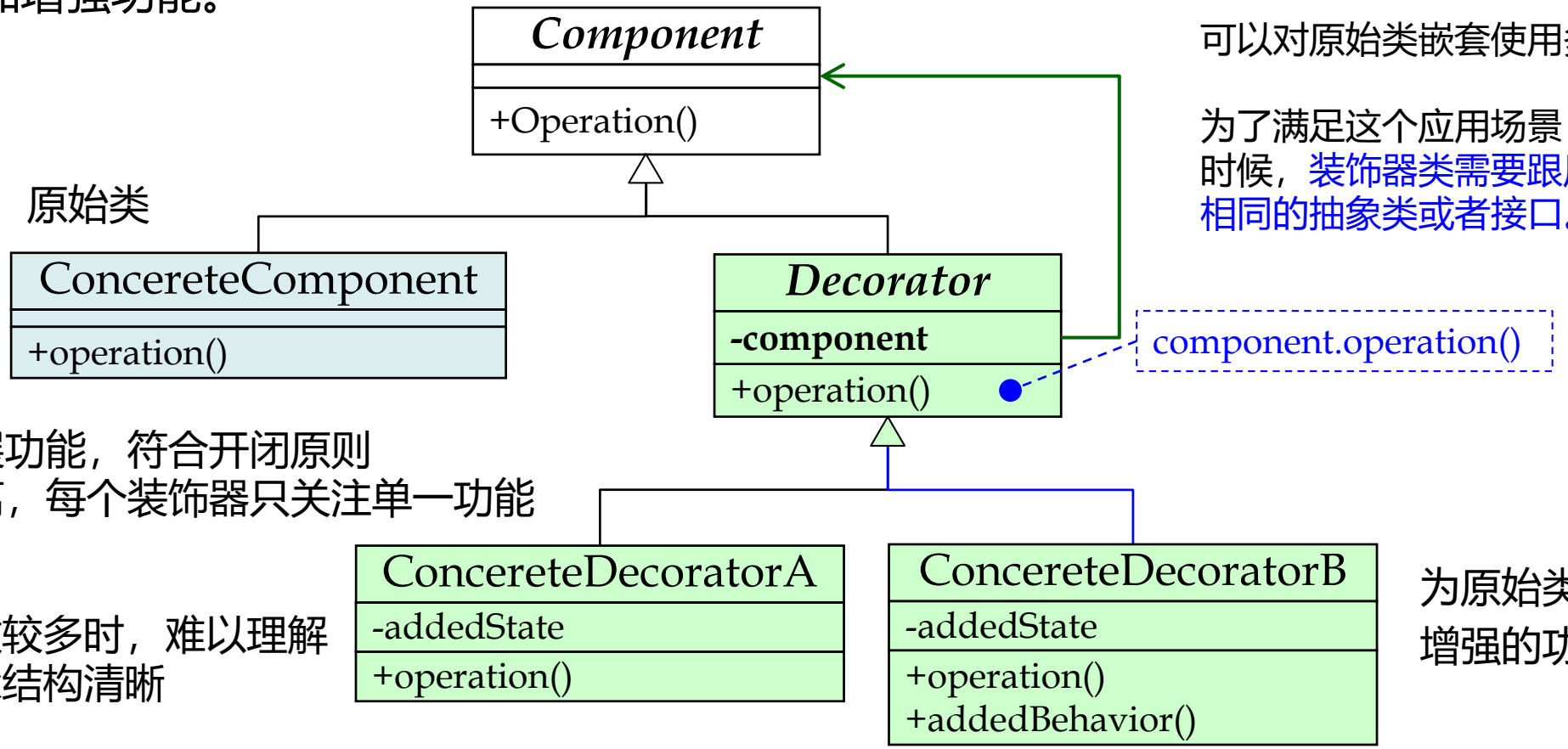
完全等同

```java
public class Client {
    public static void main(String[] args) {
        Beverage beverage = new Whip(new Mocha(new DarkRoast()));
        System.out.println(beverage.getDescription() + " $" + beverage.cost());
    }
}
```

```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.4.1\lib\idea_rt.jar=8546" -Dfile.
Dark Roast Coffee, Mocha, Whip $1.29
```

3.3 装饰器模式（Decorator Design Pattern）概念与结构

- 装饰器模式主要解决继承关系过于复杂的问题，通过组合来替代继承。它主要的作用是给原始类添加增强功能。



可以对原始类嵌套使用多个装饰器。

为了满足这个应用场景，在设计的时候，装饰器类需要跟原始类继承相同的抽象类或者接口。

优点：

灵活扩展功能，符合开闭原则
职责分离，每个装饰器只关注单一功能

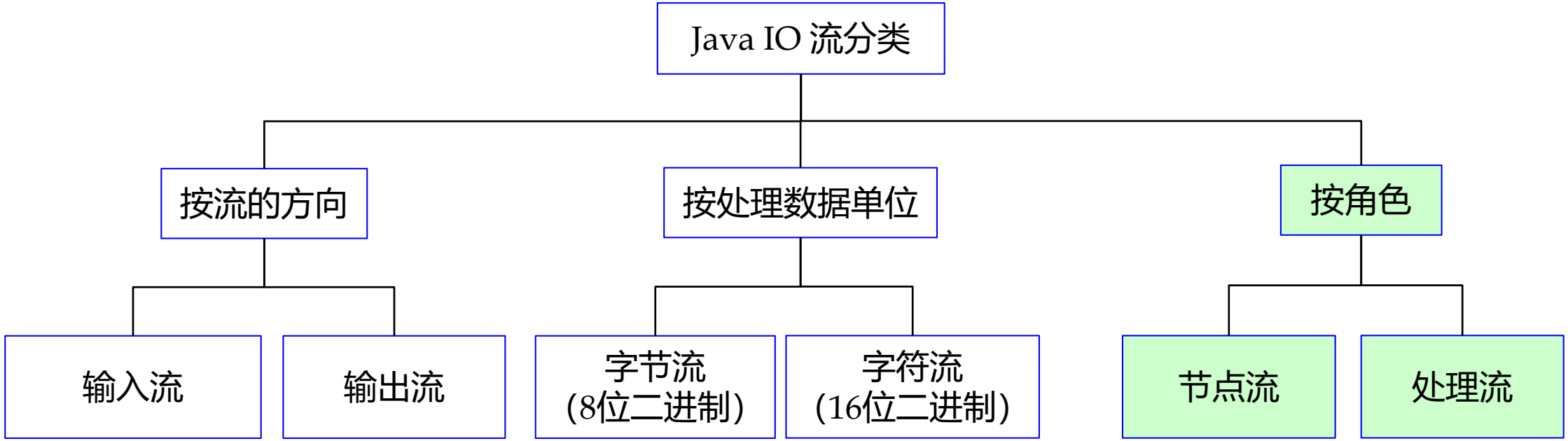
缺点：

包裹层数较多时，难以理解
没有继承结构清晰

为原始类添加的增强的功能

3.4 装饰模式在 Java IO 中的应用

- 在 Java 开发环境中，主要是由包 java.io 中提供的一系列的类和接口来实现输入/输出处理

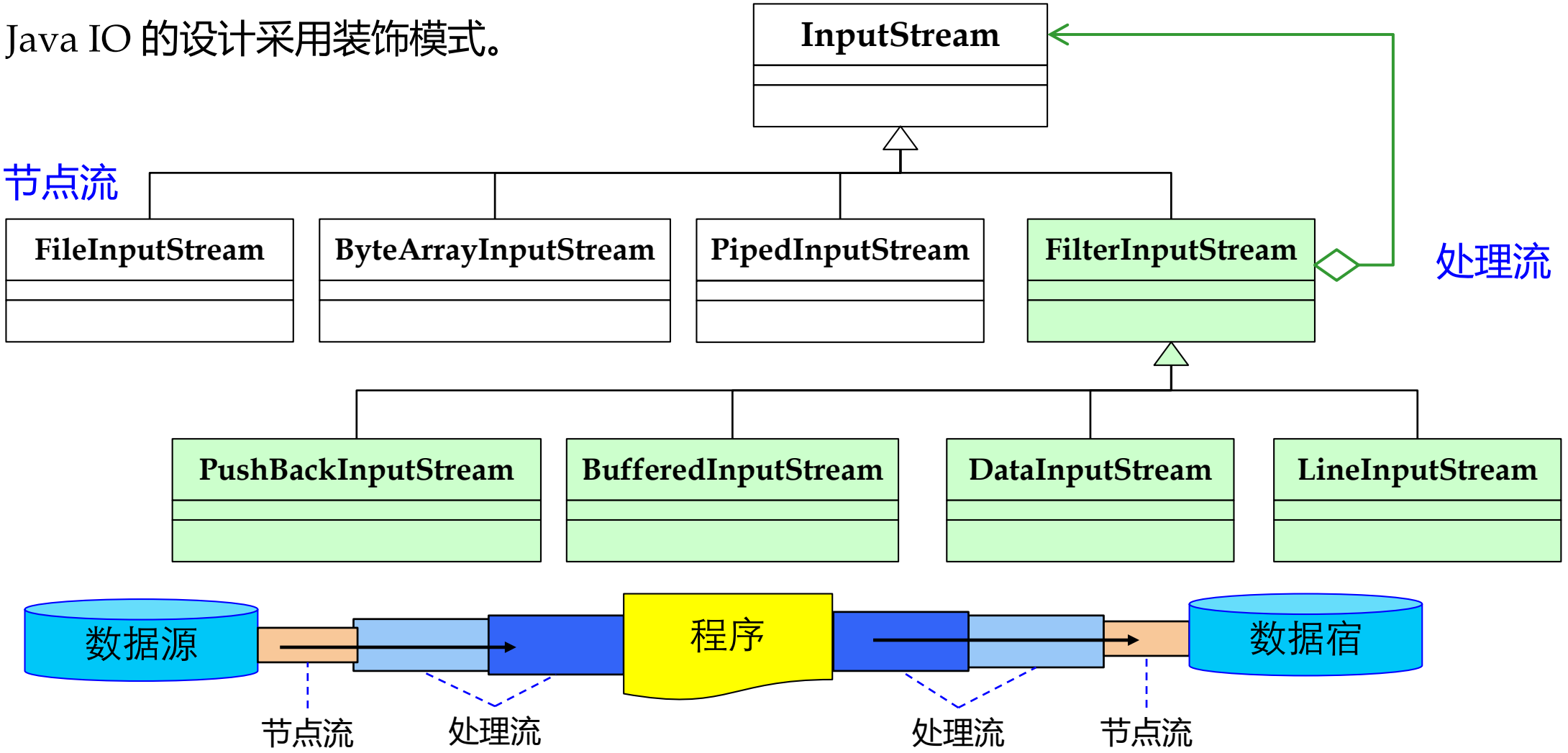


节点流：程序直接连接到数据源和数据宿，进行读写。

处理流：又称高级流或包装流，处理流对一个已存在的流进行连接，通过封装后的流来进行读写。

3.4 装饰模式在 Java IO 中的应用

- Java IO 的设计采用装饰模式。



3.4 装饰模式在 Java IO 中的应用

```
FileInputStream fis = new FileInputStream(“1.mp3”);
FileOutputStream fos = new FileOutputStream(“1copy.mp3”);
int b;
while ((b = fis.read()) != -1) { // 在不断的读取每一个字节
     fos.write(b); // 将每一个字节写出
 }
fis.close(); // 关流释放资源
fos.close();
```

仅使用节点流进行文件拷贝

```
BufferedInputStream bis = new BufferedInputStream(new FileInputStream("1.mp3"));
BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream("1copy.mp3"));
int b;
while ((b = bis.read()) != -1) {
    bos.write(b);
}
bis.close();
bos.close();
```

对节点流增加缓冲功能

3.5 装饰模式练习

- 《西游记》第六回 “观音赴会问原因 小圣施威降大圣”中对二郎神和孙悟空的决斗有精彩的描写：
  - 大圣慌了手脚，…，摇身一变，变作个麻雀儿，飞在树梢头钉住…
  - 二郎见大圣变了麻雀儿，…就摇身一变，变作个饿鹰儿，抖开翅，飞将去扑打,大圣见了，变作一只大鹞老，冲天而去。二郎见了，变作一只大海鹤,大圣又将身按下，入涧中，变作一个鱼儿…
  - 二郎赶至涧边,变作个鱼鹰儿,大圣就撺出水中，变作一条水蛇，游近岸，钻入草中
  - …
- 可能用到的词汇如下，不足之处自行添加，也可自行设计词汇表。
  - 齐天大圣（monkey king），大圣本身（monkey），大圣化身（transformations），麻雀（sparrow），鹰（eagle），鱼儿（fish），水蛇（snake）
- 要求①绘制孙悟空能够变身的框架类图，满足开闭原则；②编写代码实现类图；③编写测试类实现孙悟空的变身过程

4. 桥接模式

Bridge Design Pattern

4.1 引例









- 冰激凌销售机售卖冰激凌时，需要根据杯子大小与冰激凌品种给出价格。设计该冰激凌销售机的定价系统。
- 杯子大小暂时分为：中杯 (Medium Cup) 和大杯 (Super Cup)
- 冰激凌品种目前有三种口味：哈根达斯 (Haagen-Dazs)，雀巢 (Nestle)，冰雪皇后 (Daily Queen)

4.2 解决方案讨论：设计方案1-扁平化的设计

SuperHaagenDazs
+getPrice():float

SuperNestle
+getPrice():float

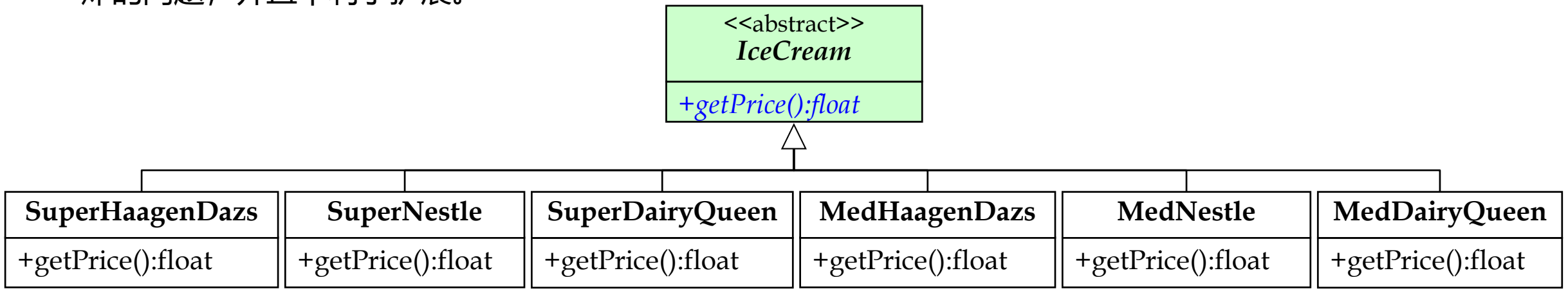
SuperDairyQueen
+getPrice():float

MedHaagenDazs
+getPrice():float

MedNestle
+getPrice():float

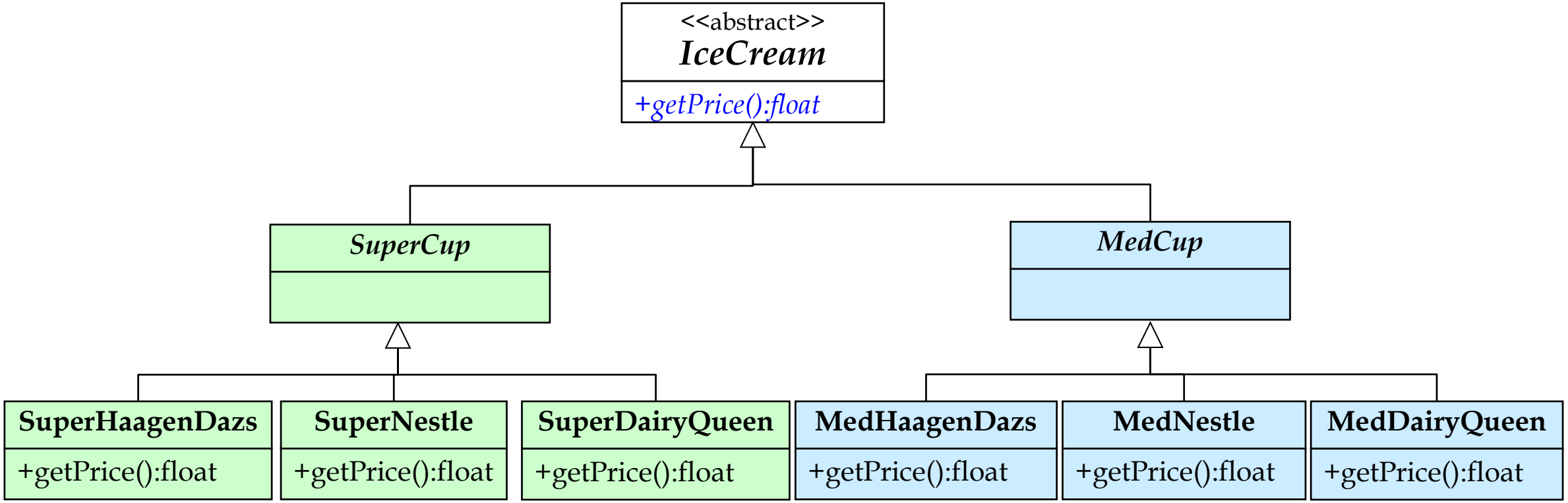
MedDairyQueen
+getPrice():float

首先能够想到设计具体类。通过组合杯子大小和咖啡品种形成各个具体类。但这种设计方式容易造成类爆炸的问题，并且不利于扩展。



因此，根据面向对象课程的基本常识，设计成层次类。这样设计符合开闭原则，但还是未解决类爆炸的问题

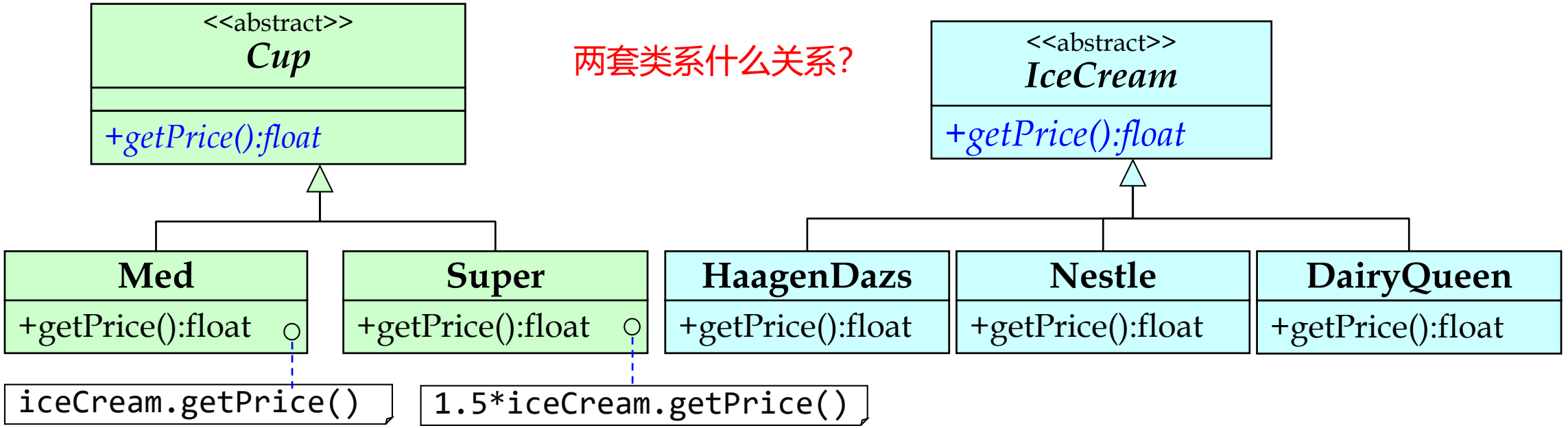
4.2 解决方案讨论：设计方案2-增加新的层次，避免扁平化



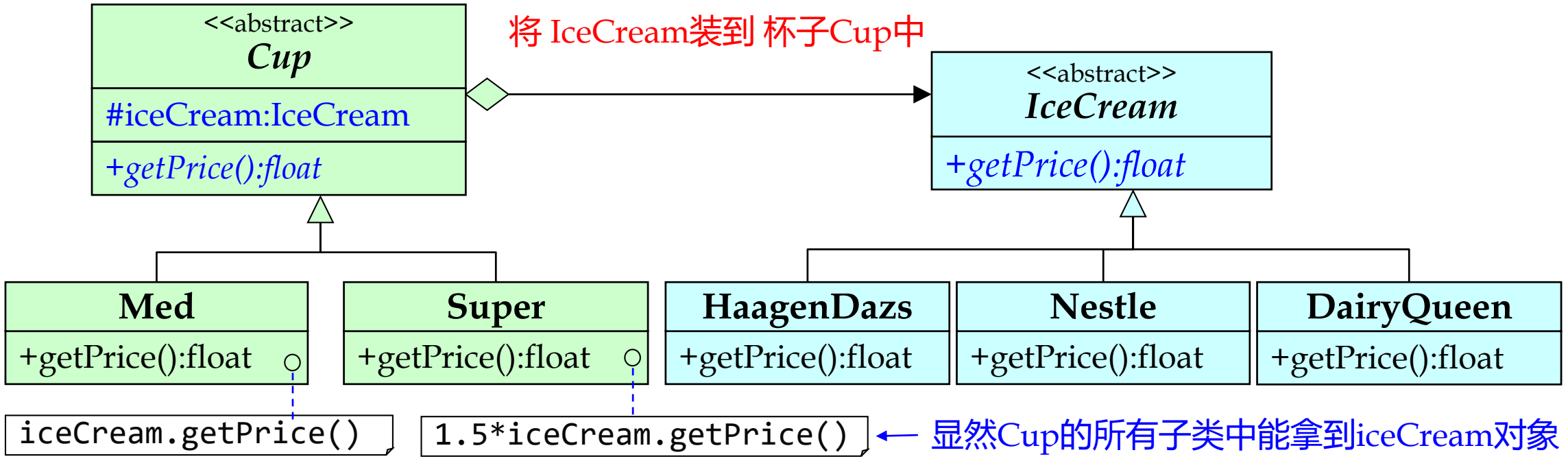
符合开闭原则，但还是比较麻烦:
(1) 增加一个冰激凌品种，需要同时在两个地方添加子类;
(2) 新增一种杯子规格（例如Small Cup），需要再增加一个层次类

4.2 解决方案讨论：设计方案3-将两个变化维度分开

- 既然杯子 Cup 和 冰激凌 IceCream 都发生变化，能否让他们各自变化，不要融合在一起?
- Cup形成一套类系， IceCream 形成一套类系?



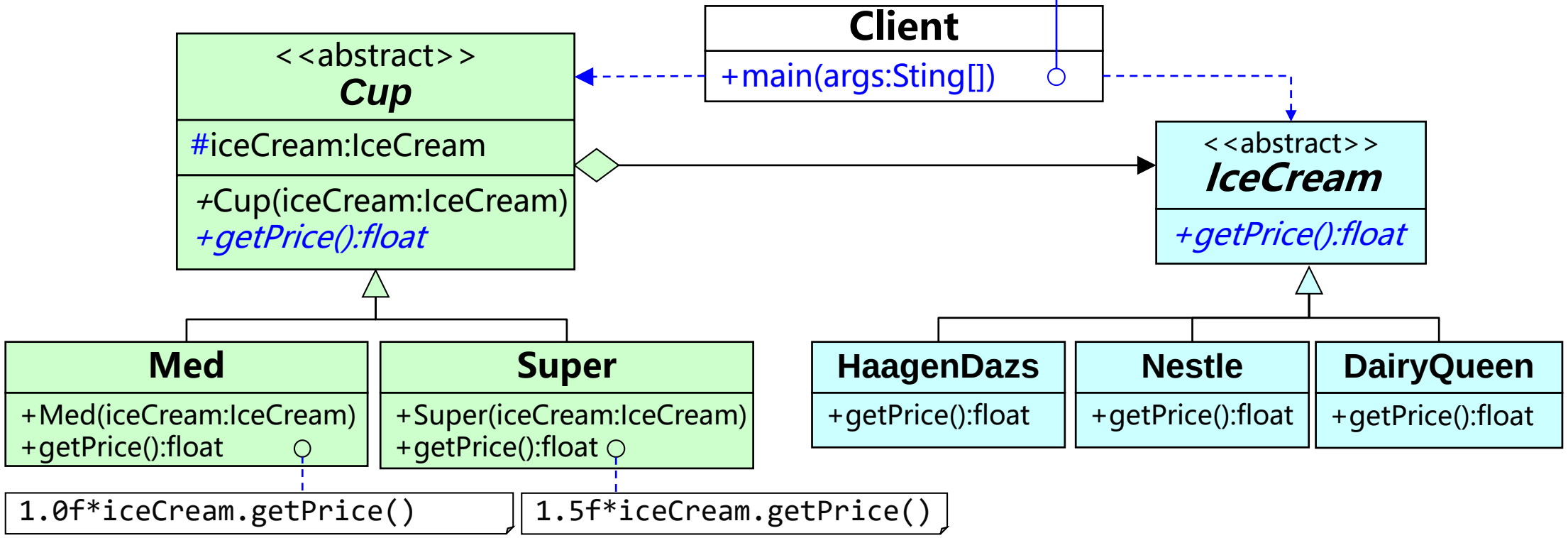
4.2 解决方案讨论：设计方案3-将两个变化维度分开



将两个变化的维度分开设计，可以使各维度独立地，互不影响地增加类，并满足开闭原则　这就是桥接模式

4.2 解决方案讨论：设计方案3-将两个变化维度分开

```
IceCream iceCream = new DairyQueen();
Cup cup = new MidCup(iceCream);
float price = cup.getPrice();
```



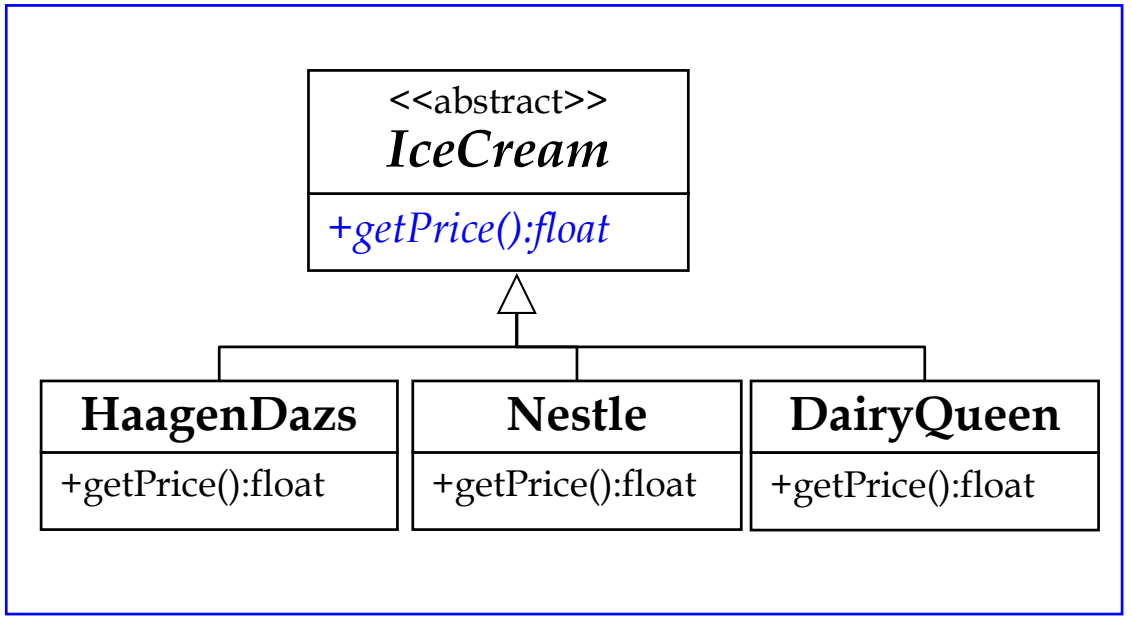
4.2 解决方案讨论：设计方案3-将两个变化维度分开

```java
public class DairyQueen extends IceCream{
    @Override
    public float getPrice() {
        return 5.0F;
    }
}
```

```java
public class HaagenDazs extends IceCream {
    @Override
    public float getPrice() {
        return 6.0F;
    }
}
```

```java
public class Nestle extends IceCream{
    @Override
    public float getPrice() {
        return 5.5F;
    }
}
```

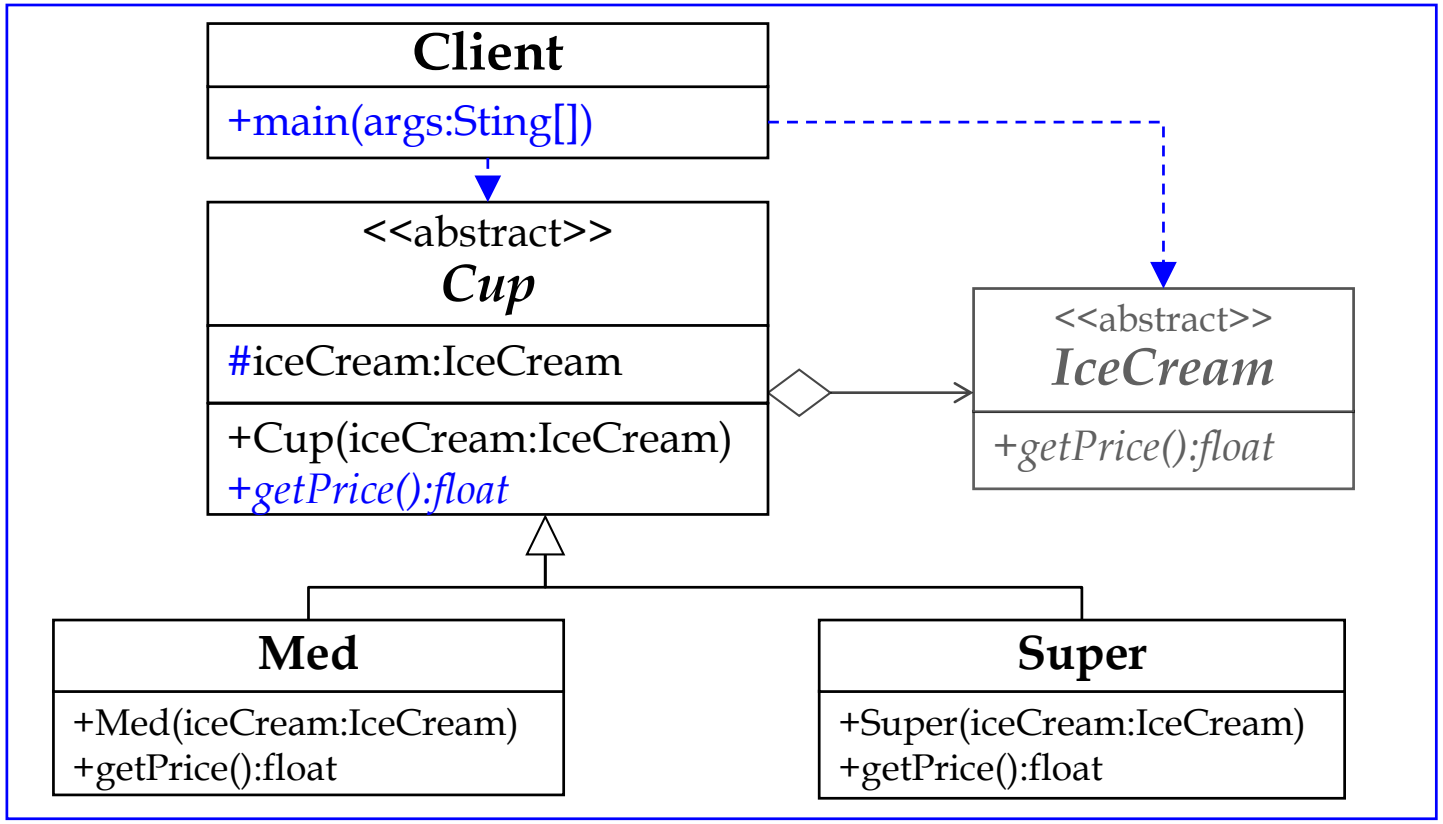
```java
public abstract class IceCream {
    public abstract float getPrice();
}
```



```java
public abstract class Cup {
    protected IceCream iceCream;
    public Cup(IceCream iceCream) {
        this.iceCream = iceCream;
    }
    public abstract float getPrice();
}
```

```java
public class SuperCup extends Cup{
    public SuperCup(IceCream iceCream) {
        super(iceCream);
    }
    @Override
    public float getPrice() {
        return 1.5f * iceCream.getPrice();
    }
}
```

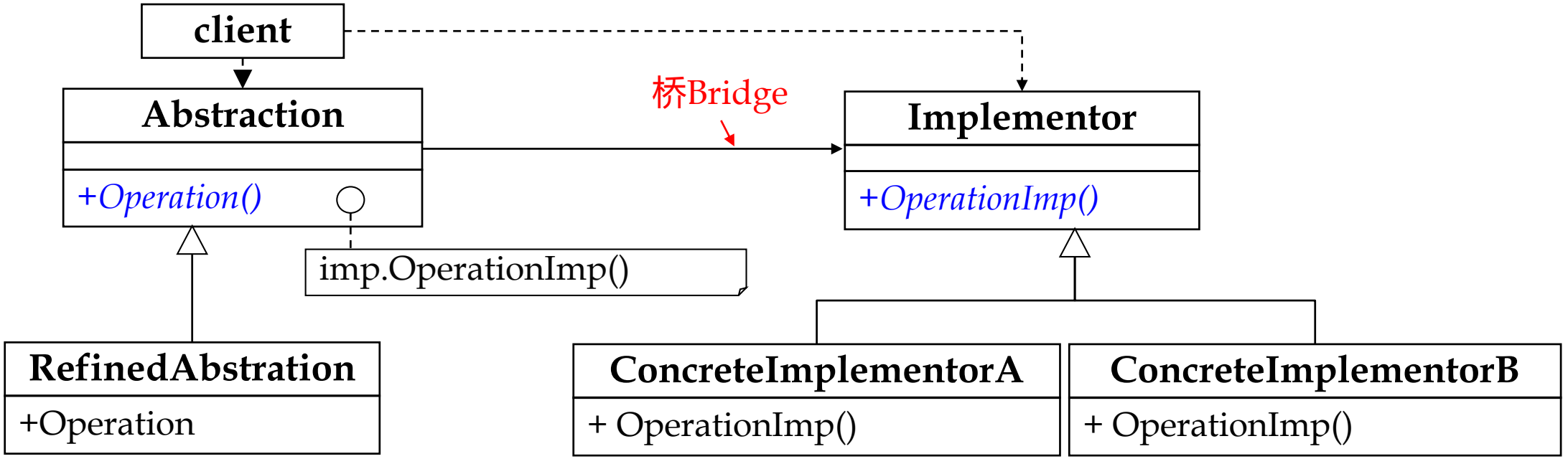
```java
public class MidCup extends Cup{
    public MidCup(IceCream iceCream) {
        super(iceCream);
    }
    @Override
    public float getPrice() {
        return iceCream.getPrice();
    }
}
```




```java
public class Client {
    public static void main(String[] args) {
        IceCream iceCream = new DairyQueen();
        Cup cup = new MidCup(iceCream);
        System.out.println("Midcup"+"DairyQueen"+cup.getPrice());
        cup = new SuperCup(iceCream);
        System.out.println("Supercup"+"DairyQueen"+cup.getPrice());
    }
}
```

4.3 桥接模式的理论

- 一个类存在两个（或多个）独立变化的维度，我们通过分离/组合的方式，让这两个（或多个）维度可以独立进行扩展。



- 扩展性：两个维度都可以添加子类，扩展系统的功能。
- 可维护性：修改一个维度，不会对另一部分产生影响，易于维护。

满足OCP

4.3 桥接模式的理论





4.4 练习：API 接口监控发送告警信息组件的重构

- API 接口监控告警的例子：根据不同的告警规则，触发不同类型的告警。

告警支持多种通知渠道，包括：

- 邮件
- 短信
- 微信
- 自动语音电话。

通知的紧急程度有多种类型，包括：

- SEVERE（严重）
- URGENCY（紧急）
- NORMAL（普通）。

  - 不同的紧急程度对应不同的通知渠道。比如， SEVERE （严重）级别的消息会通过“自动语音电话”告知相关人员。

4.4 练习：API 接口监控发送告警信息组件的重构

给出的原始代码

```
public class Notification { // 3个属性的setter省略。无getter
    private List<String> emailAddresses;
    private List<String> telephones;
    private List<String> wechatIds;

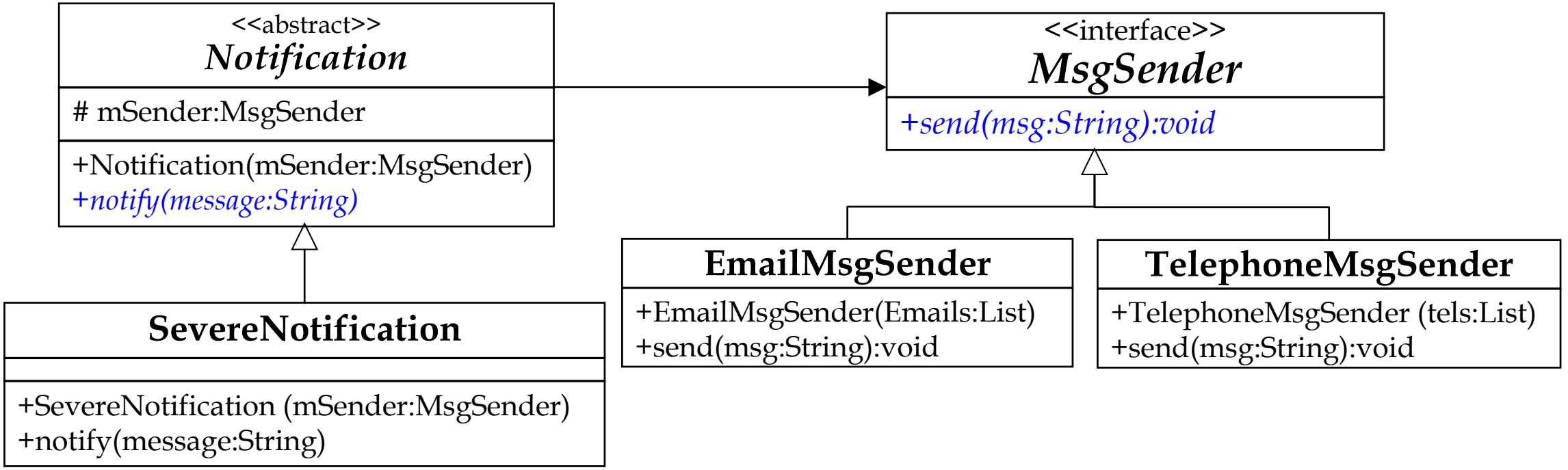
    public void notify(NotificationEmergencyLevel level, String message) {
        if (level.equals(NotificationEmergencyLevel.SEVERE)) {
            //...自动语音电话 代码略
        } else if (level.equals(NotificationEmergencyLevel.URGENCY)) {
            //...发微信 代码略
        } else if (level.equals(NotificationEmergencyLevel.NORMAL)) {
            //...发邮件 代码略
        }
}
```

```
public enum NotificationEmergencyLevel {
    SEVERE, URGENCY, NORMAL
}
```

4.4 练习：API 接口监控发送告警信息组件的重构

- 重构要求：
  - 充分考虑通知渠道和紧急程度都可能膨胀
  - 绘制出重构后的类图，满足 OCP
  - 根据类图编写框架代码

4.4 练习：API 接口监控发送告警信息组件的重构



UrgencyNotification 和 NormalNotification 类图与 SevereNotification 基本一致，略

4.4 练习：API 接口监控发送告警信息组件的重构

```
public interface MsgSender {
    void send(String message);
}
```

```
public class TelephoneMsgSender implements MsgSender {
    private List<String> telephones;

    public TelephoneMsgSender(List<String> telephones) {
        this.telephones = telephones;
    }

    @Override
    public void send(String message) {
        //...
        System.out.println("send telephones message");
    }
}
```

4.4 练习：API 接口监控发送告警信息组件的重构

```
public abstract class Notification {
    protected MsgSender msgSender;
    public Notification(MsgSender msgSender) {
        this.msgSender = msgSender;
    }
    public abstract void notify(String message);
}
```

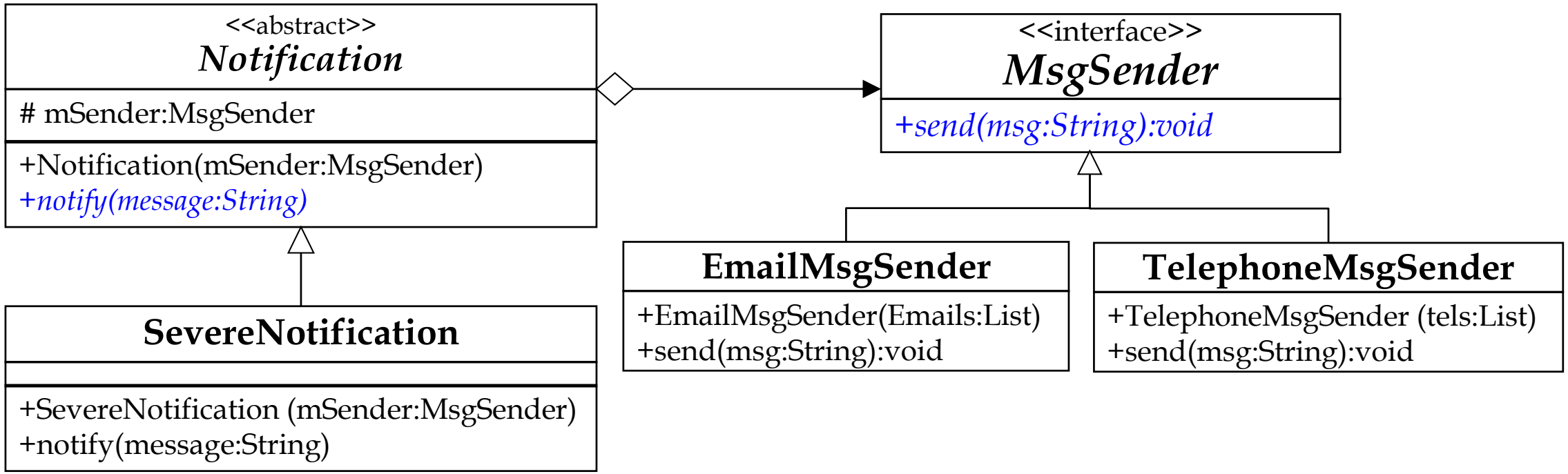
```
public class SevereNotification extends Notification {
    public SevereNotification(MsgSender msgSender) {
        super(msgSender);
    }
    @Override
    public void notify(String message) {
        msgSender.send(message);
    }
}
```

4.4 练习：API 接口监控发送告警信息组件的重构

```
public class Client {
    public static void main(String[] args) {
        List<String> telephones = new ArrayList<>();
        telephones.add("13163112345");
        telephones.add("12456234567");
        MsgSender msgSender = new TelephoneMsgSender(telephones);
        Notification notification = new SevereNotification(msgSender);
    }
}
```

4.4 练习：API 接口监控发送告警信息组件的重构

- 疑问：未实现题目要求的发送方式和紧急程度绑定。例如 severe 时，必须用电话发送？



Notification类系和 MsgSender 类系两者可以独立开发，通过组合关系（也就是桥梁）任意组合在一起。不同紧急程度的消息和发送渠道之间的对应关系，不是在代码中固定写死的，我们可以动态地去指定（比如，通过读取配置来获取对应关系）。

本章总结

模式	核心思想	关键特征
适配器	两个不兼容的接口之间互联互通	适配器可以提供跟原始类不同的接口，而代理模式、装饰器模式提供的都是跟原始类相同的接口。
代理	不改变原始类的情况下，为其附加功能	主要目的是控制访问，而非加强功能，这是它跟装饰器模式最大的不同
装饰器	装饰者模式在不改变原始类接口的情况下，对原始类功能进行增强	装饰器和被装饰对象实现相同接口，可递归嵌套，运行时动态组合。
桥接	将抽象部分与实现部分分离，使两者能够独立变化	各维度可独立扩展，通常用于处理多维度变化的系统。

组合模式 Composite Pattern、享元模式 Flyweight Pattern、外观模式 Facade Pattern 请自行学习。

The end

November 30, 2025